Why Use the Math SDK?

Traditionally, developing slot games involves navigating complex mathematical models to balance payouts, hit rates, and player engagement. This process can be time-consuming and resource-intensive. The Carrot Math SDK eliminates these challenges by providing:

- Predefined Frameworks: Start with customizable templates or sample games to accelerate development.
- Mathematical Precision: Simulate and optimize win distributions using discrete outcome probabilities, ensuring strict control over game mechanics.
- Seamless Integration: Outputs are formatted to align with the Carrot RGS, enabling quick deployment to production environments.
- Scalability: Built-in multithreading and optimization tools allow for efficient handling of large-scale simulations.

Who Is This For?

The Carrot Math SDK is ideal for developers looking to:

- Create custom slot games with unique mechanics.
- Optimize game payouts and hit rates without relying on extensive manual calculations.
- Generate detailed simulation outputs for statistical analysis.
- Publish games on Stake.com with minimal friction.

Static File Outputs

Physical slot-machines (and many of those used in iGaming) generate results in real time by programming game-logic onto the RGS/backend. When a game is requested, a cryptographically secure random number generator selects a random reel-stop position for every active reel, and the game-logic flows from the starting board position. The drawbacks of this method is that since a single reel-strip could easily have 100+ symbols, with typically 5 reels, there are 100^5 (10 billion) unique board combinations.Explicitly calculating game payouts or Return to Player (RTP) is often infeasible, so extensive simulations are used to estimate outcomes. Stake Engine requires all game-outcomes to be known at the time of publication. Storing instructions for all possible game outcomes is impractical. Instead, a subset of results is used to define the game.

These outputs are broken up into two main parts: 1. game logic files and 2. CSV payout summaries. The game-logic files contain an ordered list of critical game details such as symbol names, board positions, payout amounts, winning symbol positions etc… Accompanying each simulation detailed in the game logic files is a CSV entry listing the simulation number, probability of selection, and payout amount. So upon a game round request, the RGS will consult the CSV/lookup table to select a simulation number, then return a JSON response from the game-logic file for this simulation number to the frontend, telling the web-client what to render, while also updating the players wallet with the payout amount. Breaking up these two files also allows us to exactly calculate the games RTP and essential win-distribution statistics at time of publication.

Get Started Today

Dive into the technical details and explore how the Carrot Math SDK can transform your game development process. With powerful tools, sample games, and detailed documentation, you’ll have everything you need to create engaging and mathematically sound games.

Setup and installation

Running the math-sdk requires Python3 and PIP to be installed!

Rust/Cargo must also be installed for the optimization algorithm to run!

Clone the Math SDK repository to get started

```
git@github.com:StakeEngine/math-sdk.git
```

Makefile (recommended)

Assuming Make and a recent version of Python3 is installed on your machine, the easiest method of setting up the SDK is using the terminal to invoke:

```
make setup
```

This will setup and activate a Python virtual environment, installing all necessary packages as defined within *requirements.txt*, and install an editable math-sdk module.

Once the relavent parameters are set for a particular game, execute the run.py file using:

```
make run GAME=&lt;game_id&gt;
```

Installing Cargo (Only if using Optimization Algorithm)

If the optimization algorithm is being utilized, Rustand Cargo should be installed.

```
curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh
```

Manual installation

*Note: This installation is for Mac operating systems, Windows OS uses the prefix python (instead of python3)

Create and Activate a Virtual Environment

It's recommended to use a virtual environment to manage dependencies. Using the Virtual Environment manager (*venv*), install Python version >=3.12 using:

```
python3 -m venv env
```

If you are using Mac, activate the env with:

```
source env/bin/activate
```

If using a Windows computer use:

```
env\Scripts\activate.bat
```

Install Dependencies

Use `pip` to install dependencies from the `requirements.txt` file:

```
python3 -m pip install -r requirements.txt
```

Install the Package in Editable Mode

Using the `setup.py` file, the package should be installed it in editable mode (for development purposes) with the command:

```
python3 -m pip install -e .
```

This allows modifications to the package source code to take effect without reinstallation.

Verify Installation

You can check that the package is installed by running:

```
python3 -m pip list
```

or testing the package import in Python:

```
python
>>> import your_package_name
```

Deactivating the Virtual Environment

When finished, deactivate the virtual environment with:

```
deactivate
```

Running your first game

There are several example games provided within `/games/`, showing how common slot mechanics may be implemented. As an example let's look at `games/0_0_lines/`, a 3-row, 5-reel game paying on 20 win-lines. Wins involving 3 or more like symbols will award an amount described by `GameConfig.(paytable/payines)`.

Run-file

Simulation parameters including number of simulations, payout statistics, optimization conditions, which modes to run etc.. are all handled within `run.py`.

Using the default settings, running:

```
make run GAME=0_0_lines
```

(or calling the script manually after activating your virtual-environment)

```
python3 games/0_0_lines/run.py
```

will output all the files required by the RGS. All required files to publish math results are found within the `library/publish_files/` folder. Even if this math-sdk is not being used to generate math results, the *books*, *lookup-tables* and *index*file are required for publication.

Testing Game Outputs

To see example output files in human-readable form, lets simulate 100 results without compression in order to inspect the JSON output, we can alter the following variables within `run.py`:

```
num_threads = 1
compression = False

num_sim_args = {
    "base": 100,
    "bonus": 100,
}

run_conditions = {
    "run_sims": True,
    "run_optimization": False,
    "run_analysis": False
}
```

When setting `num_sim_args`, we are essentially running the function `run_spin()` within `gamestate.py` 100 times, with simulation criteria being assigned within the `GameConfig()` class. We can see which criteria (basegame, 0-wins, feature games, max-wins etc..) have been applied to which simulation within the `library/lookup_tables/lookUpTableIdToCriteria_<mode>.csv`file. Here, we will not run the optimization or analysis because 100 results does not give a large enough range of results to approach large-scale statistics. We only need 1 CPU thread, so we can change this from 10 to 1 since it should only a take a second or two to run.

Inspecting the output file, `library/books/books_base.jsonl` shows each simulation, identified by `id` (1-100). Each simulation-id has an `events` tag, which communicates to the front-end framework which symbols are revealed, win positions and amounts, and any of game-specific logic. Each simulation has a `payoutMultiplier` value which is the final payout amount for that round. This value directly corresponds to the value in `library/lookup_tables/lookUpTable_base.csv`. When a round-response is returned from by the RGS from the `play/` API, it is the contents of the `events` tag which is returned in the response body.

If we look at the results for, say, simulation 58:


```
{
    "id": 58,
    "payoutMultiplier": 10,
    "events": [
        {
            "index": 0,
            "type": "reveal",
            "board":[...],
            "paddingPositions": [...],
            "gameType": "basegame",
            "anticipation": [...]
        },
        {
            "index": 1,
            "type": "winInfo",
            "totalWin": 10,
            "wins": [
                {
                    "symbol": "L5",
                    "kind": 3,
                    "win": 10,
                    "positions": [...],
                    "meta": {}
                }
            ]
        },
        {
            "index": 2,
            "type": "setWin",
            "amount": 10,
            "winLevel": 2
        },
        {
            "index": 3,
            "type": "setTotalWin",
            "amount": 10
        },
        {
```

```
            "index": 4,
            "type": "finalWin",
            "amount": 10
        }
    ],
    "criteria": "basegame",
    "baseGameWins": 0.1,
    "freeGameWins": 0.0
}
```

This tells up what board-symbols to reveal, the winning positions on this board, the payout amount, and sets the win counters. If we now open the lookup-table file and search for simulation number 58 we see the result: `58,1,10`, matching what was given to us within the `books` file. Note that all simulations are initially given a selection weight of `1` (the second value in each CSV row). The optimization program is what sets these weights to ensure that the game-mode is balanced to a specified RTP.

Larger simulation batches

When starting with a new game, it is suggested to start by running a small number of simulations saved in uncompressed JSON format for debugging. Once satisfied with the gamestate output, larger simulations should be run. For a production-ready game it is typically recommended to run 100k+ simulations per mode to ensure that there is a diverse range of payout multipliers to optimize over, and to significantly reduce the chances of any single player receiving the same round result more than once. We set the following parameters indicating that we want to use 20 threads for simulating the game-logic for 10,000 simulations per mode, output in compressed (.json.zst) format, we will then use 20 threads when running the optimization algorithm (this will produce modified lookup-tables such as `lookUpTable_base_0.csv`).

```
num_sim_args = {
    "base": int(1e4),
    "bonus": int(1e4),
}

run_conditions = {
    "run_sims": True,
    "run_optimization": True,
```

```
    "run_analysis": True,
    "upload_data": False,
}
```

In the terminal you should seethe game RTP printed out as each thread finishes

```
Thread 0 finished with 1.632 RTP. [baseGame: 0.043, freeGame: 1.588]
```

Flor the `bonus` mode, this is telling us that thread 0/10 finished with a total RTP of 163.2%, with 4.3% coming from the basegame (wins on the reveal of Scatter symbols), and 158.8% RTP coming from freegame wins. This is higher than our expected 97%, though we are forcing significantly more max-win simulations than will naturally be awarded, so this is okay. The optimization algorithm will adjust these weights to balance the game properly.

By setting `run_analysis: True` we are indicating that we would like to generate a PAR sheet, summarizing key game statistics and hit-rates. This program will use the `library/lookup_tables/lookUpTableSegmented_<mode>.csv`file to determine which game-types contributed to the final round wins, in conjunction with the pay-table and `library/forces/force_record_<mode>.json`files to generate frequency and average-win statistics for specific events or win combinations.

Next steps

These outputs corresponding directly with example Storybook packages within the `web-sdk`. It is recommended to take look through this pack to see how these math events are passed and displayed on the frontend. If you have your own game in mind you can use one of the sample games provided as a template and implement your own unique rules within the `games/<game_name>/` directory. You will likely need to specify configuration values for things like multipliers, prize-value etc.. wtihin `game_config.py`. Then any unique calculations and events should be handled within the games `game_executables/game_calculation` files. Generally speaking, reusable functions,

events or calculation should like with `/src/`, which one-off game functionality belongs within that games folder `/games/<game_id>/`.

Math verification

When uploading static math files to the RGS, Stake Engine will carry out preliminary checks to ensure ensure game-logic is of the expected format. The corresponding payout multipliers and probabilities are analyzed as a means of providing a quick summary of game statistics on the backend.

Minimum file requirements

For a game with one game-mode, there will be 3 files required for the Math to be published successfully.

- Index file (must be called *index.json and contain the mode name, cost multiplier and logic/CSV filenames)
- Lookup table (CSV file, with each line containing ID, Probability, Payout)
- Game logic (zStandard compressed JSON-lines (__.jsonl.zst))

Index file format

When selecting a directory to upload from for the Stake Engine math there must exist a JSON-encoded file called *index.json* with the strictly enforced form:

```
{
```

```
    "modes": [
        {
            "name": <string>,
            "cost": <float>,
            "events": <string>"<logic_file>.jsonl.zst",
            "weights": <string>"<lookup_table>.csv"
        },
        ...
    ]
}
```

For example, for a game with 2-modes:

```
{
    "modes": [
        {
            "name": "base",
            "cost": 1.0,
            "events": "books_base.jsonl.zst",
            "weights": "lookUpTable_base_0.csv"
        },
        {
            "name": "bonus",
            "cost": 100.0,
            "events": "books_bonus.jsonl.zst",
            "weights": "lookUpTable_bonus_0.csv"
        }
    ]
}
```

CSV format

When calculating various statistical values on the RGS side, it is much more efficient and robust to work with unsigned integer values (since no payouts or probabilities will ever be negative). This avoids misinterpreting values due to rounding or floating-point errors. For every game-round uploded within the game-logic there must a summary CSV table containing rows of `uint64`values. We require the payoutMuliplier value in the

third column to exactly match those provided in the game-logic file. There values are extracted and hashed to ensure identical `payoutMultiplier`values.

```
simulation number, round probability, payout multiplier
```

For example:

```
1,199895486317,0
2,25668581149,20
3,126752606,140
...
```

Repository Directory Overview

This repository is organized into several directories, each focusing on a specific aspect of the game creation process. Below is a breakdown of the main directories and their purposes:

Main Directories

- `games/`

    - Contains sample slot games showcasing widely used mechanics and modes:
        - `0_0_cluster`: Cascading cluster-wins game.
        - `0_0_lines`: Basic win-lines example game.
        - `0_0_ways`: Basic ways-wins example game.
        - `0_0_scatter`: Pay-anywhere cascading example game.
        - `0_0_expwilds`: Expanding Wild-reel game with an additional prize-collection feature.
- `src/`
    - Core game setup functions, game mechanics, frontend event structures, wallet management, and simulation output control. This directory contains reusable code shared across games. Edit with caution.
    - Subdirectories:
        - `calculations/`: Board and symbol setup, various win-type game logic.
        - `config/`: Generates configuration files required by the RGS, frontend, and optimization algorithm.
        - `events/`: Data structures passed between the math engine and frontend engine.
        - `executables/`: Commonly used groupings of game logic and events.
        - `state/`: Tracks the game state during simulations.
        - `wins/`: Wallet manager handling various win criteria.
        - `write_data/`: Handles writing simulation data, compression, and force files.
- `utils/`
    - Contains helpful functions for simulation and win-distribution analysis:

        - `analysis/`: Constructs and analyzes basic properties of win distributions.
        - `game_analytics/`: Uses recorded events, paytables, and lookup tables to generate hit-rate and simulation properties.
- `tests/`
    - Includes basic PyTest functions for verifying win calculations:
        - `win_calculations/`: Tests various win-mechanic functionality.
- `uploads/`
    - Handles the data upload process for connecting and uploading game files to an AWS S3 bucket for testing.
- `optimization_program/`
    - Contains an experimental genetic algorithm (written in Rust) for balancing discrete-outcome games.
- `docs/`
    - Documentation files written in Markdown.

Detailed Subdirectory Breakdown

`src/`

- `calculations/`: Handles board and symbol setup, along with various win-type game logic.
- `config/`: Creates configuration files required by the RGS, frontend, and optimization algorithm.

- `events/`: Defines data structures passed between the math engine and frontend engine.
- `executables/`: Groups commonly used game logic and events for reuse.
- `state/`: Tracks the game state during simulations.
- `wins/`: Manages wallet functionality and various win criteria.
- `write_data/`: Writes simulation data, handles compression, and generates force files.

`games/`

- `0_0_cluster/`: Sample cascading cluster-wins game.
- `0_0_lines/`: Basic win-lines example game.
- `0_0_ways/`: Basic ways-wins example game.
- `0_0_scatter/`: Pay-anywhere cascading example game.
- `0_0_expwilds/`: Expanding Wild-reel game with an additional prize-collection feature.

`utils/`

- `analysis/`: Constructs and analyzes basic properties of win distributions.
- `game_analytics/`: Generates hit-rate and simulation properties using recorded events, paytables, and lookup tables.

`tests/`

- `win_calculations/`: Tests various win-mechanic functionality.

`uploads/`

- Handles the process of uploading game files to an AWS S3 bucket for testing.

`optimization_program/`

- Experimental genetic algorithm (written in Rust) for balancing discrete-outcome games.

The State Machine

Introduction

The GameState class serves as the central hub for managing all aspects of a simulation batch. It handles:

- Simulation parameters
- Game modes
- Configuration settings
- Simulation results
- Output files

The entry point for all game simulations is the `run.py` file, which initializes parameters through the config class and creates a GameState object. The GameState ensures

consistency across simulations and provides a unified structure for managing game logic and outputs.

Key Responsibilities of `GameState`

Simulation Configuration

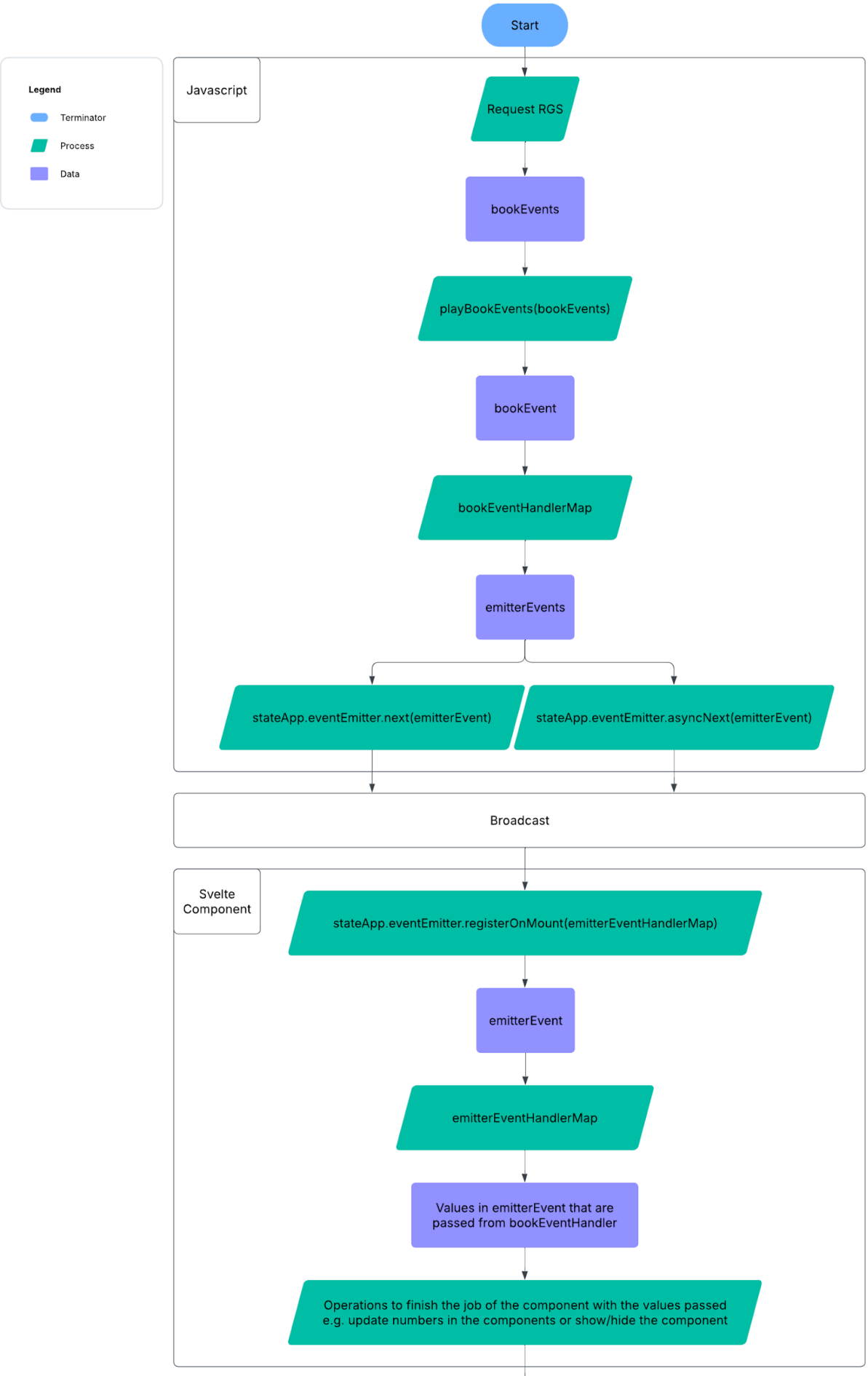
- Compression
- Tracing
- Multithreading
- Output files
- Cumulative win manager

Game Configuration

- Betmode details (costs, names, etc.)
- Paytable
- Symbols
- Reelsets

These global `GameState` attributes remain consistent across all game modes and simulations. When a simulation runs, the `run_spin()` method creates a sub-instance of the GeneralGameState, allowing modifications to game data directly through the `self` object. This design reduces the need for passing objects between functions, streamlining game logic development.

At a high level, the structure of the engine is shown below


Legend
Terminator
Process
Data
Start
Javascript
Request RGS
bookEvents
playBookEvents(bookEvents)
bookEvent
bookEventHandlerMap
emitterEvents
stateApp.eventEmitter.next(emitterEvent)
stateApp.eventEmitter.asyncNext(emitterEvent)
Broadcast
Svelte Component
stateApp.eventEmitter.registerOnMount(emitterEventHandlerMap)
emitterEvent
emitterEventHandlerMap
Values in emitterEvent that are passed from bookEventHandler
Operations to finish the job of the component with the values passed e.g. update numbers in the components or show/hide the component

Game logic format

Round information returned through the */play* API corresponds to a single simulation outcome returned in JSON format. For efficiency, we require this data to be stored in compressed *.jsonl* format. Currently zStandard (.zst) encoding must be used, though this will be expanded upon in the near future. In order to identify simulation IDs, payouts and logic we enforce the condition that every simulation contains the key fields:

```
"id": <int>,
 "events" <list<dict>>,
 "payoutMultiplier": <int>
```

For example, at a minimum the game round, printed to *jsonl* before compression will have the format:

```
{
    "id": 1,
    "events": [{}, ...],
    "payoutMultiplier": 1150
}
```

Where the payoutMultiplier value corresponds to an 11.5x payout for a base game round (costing 1.0x). The three JSON key fields: id, events, payoutMultipler are required for every round returned.

Extending Core Functionality

The `GameState` class acts as a super-class containing core functionality. Custom games can extend or override this functionality using Python's Method Resolution Order (MRO). Once simulations are complete, the relevant output files are generated

sequentially for each BetMode. These outputs can then be optimized and uploaded to the Admin Control Panel (ACP).

Class Inheritance

Why Use Class Inheritance?

Class inheritance ensures flexibility, allowing developers to access core functions while customizing specific behaviors for each game. Core functions are defined in the source files and can be overridden at the game level.

GameStateOverride (game/game_override.py)

This class is the first in the Method Resolution Order (MRO) and is responsible for modifying or extending actions from the `state.py` file. For example, all sample games override the `reset_book()` function to accommodate game-specific parameters:

```
def reset_book(self):

    super().reset_book()

    self.reset_grid_mults()

    self.reset_grid_bool()

    self.tumble_win = 0
```

GameExecutables (game/game_executables.py)

This class groups commonly used game actions into executable functions. These functions can be overridden to introduce new mechanics at the game level. For example, triggering freespins based on scatter symbols:

```
config.freespin_triggers = {3: 8, 4: 10, 5: 12}
```

```
def update_freespin_amount(self, scatter_key: str = "scatter") -> None:

    self.tot_fs =
self.config.freespin_triggers[self.gametype][self.count_special_symbols(scatter
_key)]

    fs_trigger_event(self, basegame_trigger=True, freegame_trigger=False)
```

However in the `0_0_scatter` sample game, we would instead want to assign the total spins to be 2x the number of active Scatters. Therefore we can override the function in the `GameExecutables`class:

```
def update_freespin_amount(self, scatter_key: str = "scatter"):

    self.tot_fs = self.count_special_symbols(scatter_key) * 2

    fs_trigger_event(self, basegame_trigger=basegame_trigger,
freegame_trigger=freegame_trigger)
```

GameCalculations (games/game_calculations.py)

This class handles game-specific calculations, inheriting from GameExecutables.

Books and Libraries

What is a “Book”?

A “book” represents a single simulation result, storing:

- The payout multiplier
- Events triggered during the round
- Win conditions

Each simulation generates a Book object, which is stored in a library. The library is a collection of all books generated during a simulation batch. These books are attached to the global GameState object and are used for further analysis and optimization.

Example JSON structure:

```
[
    {
        "id": int,
        "payoutMultiplier": float,
        "events": [ {}, {}, {} ],
        "criteria": str,
        "baseGameWins": float,
        "freeGameWins": float
    }
]
```

Resetting the Book

At the start of a simulation, the book is reset to ensure a clean state:

```
def reset_book(self) -> None:
    self.book = {
        "id": self.sim + 1,
        "payoutMultiplier": 0.0,
        "events": [],
        "criteria": self.criteria,
```

}

Lookup Tables

What are Lookup Tables?

Lookup tables provide a summary of all simulation payouts, offering a convenient way to calculate win distribution properties and Return To Player (RTP) values. Each table is stored as a CSV file and contains the following columns:

Simulation Number	Simulation Weight	Payout Multiplier
1	1	0.0
2	1	92.3
…	…	…

The payoutMultipler attached to a bookrepresents the final amount paid to the player, inclusive or *basegame* and *freegame* wins. The LookUpTable *csv* file is a summary of all simulation payouts. This provides a convenient way to calculate win distribution properties and Return To Player calculations. All lookup tables will be of the format:

Purpose of Lookup Tables

- Win Distribution Analysis: Analyze payout distributions across simulations.
- RTP Calculation: Calculate the overall RTP for a game mode.
- Optimization: Serve as input for the optimization algorithm, which adjusts simulation weights to achieve desired payout characteristics.

File Naming Convention

- Initial Lookup Tables: lookUpTable_mode.csv
- Optimized Lookup Tables: lookUpTable_mode_0.csv

The optimization algorithm modifies the weight values in the lookup table, which are initially set to 1. These optimized tables are then used for further analysis or deployment.

Intended Engine Usage

Game Files

As seen in the [example games](), all games follow a recommended structure, which should be copied from the games/template folder.

```
game/

├── library/

|----- books/

|----- books_compressed/
```

```
|----- configs/

|----- forces/

|----- lookup_tables/

├── reels/

├── readme.txt

├── run.py

├── game_config.py

├── game_executables.py

├── game_calculations.py

├── game_events.py

├── game_override.py

└── gamestate.py
```

Sub-folders within library/ are automatically generated if they do not exist at the completion of the simulation. readme.txt is used for developer descriptions of game mechanics and miscellaneous information relevant to that particular game.

While all commonly used engine functions are handled by classes within their respective src/ directory, every game is likely to be unique in some way and these game-files allow the user to override existing functions in order to add additional engine features to suit their use-case, or implement game-specific logic.

The game_config/executables/calculations/events/override files offer extensions on actions defined in the source files section, which should be consulted for more detailed information.

Run-file

This file is used to set simulation parameters, specifically the configuration and `GameState` classes. The required specifications include:

Parameter	Type	Description
`num_threads`	`int`	Number of threads used for multithreading
`rust_threads`	`int`	Number of threads used by the Rust compiler
`batching_size`	`int`	Number of simulations run on each thread
`compression`	`bool`	`True` for `.json.zst` compressed books, `False` for `.json` format
`profiling`	`bool`	`True` outputs and opens a `.svg` flame graph
`num_sim_args`	`dict[int]`	Keys must match bet mode names in the game configuration

All simulations are passed to the `create_books()` function which carries out all the simulations and handles file output. This function will populate `library/` `books_compressed`, `books`, `forces`, `lookup_tables` folders.

Once the simulations are completed, the **gamestate** is passed to `generate_configs(gamestate)` which handles generating config files used for the frontend (`config_fe.json`), backend (`config.json`) and [optimization](/docs/math/optimization-algorithm) (`config_math.json`).

Library Folders

books/books_compressed

Depending on the **compression** tag passed to `create_books()` the `books/` or `books_compressed/` folders will be populated with the events emitted from the simulation.

configs

This will consist of three `.json` files for the math, frontend and backend.

lookup_tables

Once any given simulation is compete the events associated are stored within the books, and the corresponding payout details are recorded in a lookup table of the format:

Simulation	Weight	Payout
`int`	`int`	`float`

All simulations start with an assigned weight of `1`, which is then modified if the optimization algorithm is applied.

Configs

The **GameConfig** inherits the **Config** class. All information defined in the *__init__* function are required inputs. Symbol information, pay-tables, reels-strips and bet-mode information are all specified here.

Gamestate

Every game has a *gamestate.py* file, where independent simulation states are handled. The *run_spin()* function is required and used as the entry_point from *create_books* to execute the a single simulation. *run_freespin* is also used in all sample games, though is not a required function if the game does not contain a free-spin entry from the base-game.

Executables

Commonly used groups of game-logic and event emission is provided in this location. Functions called in the *run_spin()* functions will typically belong to the Executables/GameExecutables classes.

Functions currently in this class include drawing random or forced game-boards, handling game-logic for several win-types and their associated win information events, updating and

Misc. Calculations

The **Executables** class inherits all miscellaneous game-logic and board-actions. Primarily this includes all win-evaluation types:

* Lines

* Ways

* Scatter (pay anywhere)

* Cluster

* Expanding wild + prize collection

Additionally other classes attached to **Executables** are tumbling/cascading of winning symbols and **Conditions** for checking the current simulation state

Standard Game Setup Requirements

Without diving into specific functions, this section is intended to walkthrough how a new slot game would generally be setup. In practice it is recommended to start with one of the sample games which closest resemble the game being made, or otherwise starting from the template.

Configuration file

Game parameters should all be set in the `GameConfig __init__()` function. This is where to set the name name, RTP, board dimensions, payouts, reels and various special symbol actions. All required fields are listed in the `Config` class and should be filed out explicitly for each new game. Next the `BetMode` classes are defined. Generally there would be at a minimum a (default) `base` game and a `freegame`, which is usually purchased.

```
class GameConfig(Config):

    def __init__(self):

        super().__init__()

        self.game_id = ""
```

```
self.provider_number = 0
self.working_name = ""
self.wincap = 0
self.win_type = "lines"
self.rtp = 0

self.num_reels = 0
self.num_rows = [0] * self.num_reels
self.paytable = {
    (kind, symbol): payout,
}

self.include_padding = True
self.special_symbols = {"property": ["sym_name"],...}

self.freespin_triggers = {
}
self.reels = {}
self.bet_modes = []
```

Each `BetMode` should likewise be set explicitly, defining the cost, rtp maximum win amounts and various gametype flags. We would like to define different win criteria within each betmode. In the sample games we define distinct criteria for any game-aspects where we would like to control either the hit-rate and/or RTP allocation. In this example we would like to control the basegame hit-rate, max-win hit-rate and freegame hit-rate.

Therefore we need to specify unique `Distribution` criteria for each of these special conditions.

```
BetMode(
    name="base",
    cost=1.0,
    rtp=self.rtp,
    max_win=self.wincap,
    auto_close_disabled=False,
    is_feature=True,
    is_buybonus=False,
    distributions=[
        Distribution(
            criteria="winCap",
            quota=0.001,
            win_criteria=self.wincap,
            conditions={
                "reel_weights": {
                    self.basegame_type: {"BR0": 1},
                    self.freegame_type: {"FR0": 1},
                },
                "force_wincap": True,
                "force_freegame": True,
            },
        ),
```

```
        Distribution(
            criteria="freegame",
            quota=0.1,
            conditions={
                "reel_weights": {
                    self.basegame_type: {"BR0": 1},
                    self.freegame_type: {"FR0": 1},
                },
                "force_wincap": False,
                "force_freegame": True,
            },
        ),
        Distribution(
            criteria="0",
            quota=0.4,
            win_criteria=0.0,
            conditions={
                "reel_weights": {self.basegame_type: {"BR0": 1}},
            },
        ),
        Distribution(
            criteria="basegame",
            quota=0.5,
            conditions={
```

```
                "reel_weights": {self.basegame_type: {"BR0": 1}},
            },
        ),
    ],
)
```

Gamestate file

When any simulation is run, the entry point will be the `run_spin()` function, which lives in the `GameState` class. `GameExecutables` and `GameCalculations` are child classes of `GameState` and also deal with game specific logic.

The generic structure would follow the format:

```
def run_spin(self, sim):
    self.reset_seed(sim) #seed the RNG with the simulation number
    self.repeat = True
    while self.repeat:
        self.reset_book() #reset local variables
        self.draw_board() #rraw board from reelstrips


        #evaluate win_data
        #update win_manager
        #emit relevant events
```

```
        self.win_manager.update_gametype_wins(self.gametype) #update cumulative
basegame wins

        if self.check_fs_condition(): #check scatter conditions

            self.run_freespin_from_base() #run freegame



        self.evaluate_finalwin()

        self.check_repeat() #Verify betmode distribution conditions are
satisfied



    self.imprint_wins() #save simulation result
```

For reproducibility the RNG is seeded with the simulation number. Betmode distribution criteria are preassigned to each simulation number, requiring the `self.repeat` condition to be initially set until the spin has completed and it can be checked that any criteria-specific conditions or win amounts are satisfied. Note that `self.repeat = False` is set in the `self.reset_book()` function. This function will reset all relevant `GameState` properties to default values.

Generally the first steps will be to use the reelstrips provided in the configuration file to draw a board from randomly chosen reelstop positions. Wins are evaluated from one of the provided win-types for the active board, and the wallet manager is updated. After this game-logic is completed the relevant events (such as `reveal` and `winInfo`) are emitted. All sample games follow these three steps:

1. Calculate current state of the board
2. Update wallet manager
3. Emit events

To keep track of which gametype wins are allocated, the wallet manger is again invoked once all basegame actions are complete. If the game have a freegame mode and the triggering conditions are satisfied the `run_freespin()` function is invoked. This mode will have a similar structure:

```
def run_freespin(self):

    self.reset_fs_spin() #reset freegame variables

    while self.fs < self.tot_fs: #account for multiple freegame spins

        self.update_freespin() #update spin number and emit event

        self.draw_board() #draw a new board using freegame reelstrips



        #evaluate win_data

        #update win_manager

        #emit relevant events



        if self.check_fs_condition(): #check retrigger conditions

            self.update_fs_retrigger_amt()



        self.win_manager.update_gametype_wins(self.gametype) #update cumulative
freegame win amounts



    self.end_freespin() #emit event to indicate end of freegame
```

While it is possible to perform all game actions within these functions, for clarity functions from `GameExecutables` and `GameCalculations`are typically invoked and should be created on a game-by-game basis depending on requirements.

Runfile

Finally to produce simulations, the `run.py` file is used to create simulation outputs and config files containing game and simulation details.

```
if __name__ == "__main__":


    num_threads = 1

    rust_threaeds = 20

    batching_size = 50000

    compression = False

    profiling = False


    num_sim_args = {

        "base": int(10),

        "bonus": int(10),

    }


    config = GameConfig()

    gamestate = GameState(config)


    create_books(

        gamestate,

        config,

        num_sim_args,
```

```
        batching_size,
        num_threads,
        compression,
        profiling,
    )
    generate_configs(gamestate)
```

The `create_books` function handles the allocation of win criteria to simulation numbers, output file format and multi-threading parameters.

Outputs

Simulation outputs are placed in the `game/library/` folder. `books/books_compressed` is the primary data-file containing all events and payout multipliers. `lookup_tables` hold the summary simulation-payout values in `.csv` format which is consumed by the optimization algorithm. Additionally for game analysis, lookup table mapping of which simulations belong to which win criteria and which gametype wins arise from are produced. `force/`file outputs contain all information used by the `.record()` function, which is again useful for analyzing the frequency and average win amounts for specific events. The optimization algorithm also uses the recorded `force` data to identify which simulations correspond to specific win criteria. Finally `config/` files contain information required by the frontend such as symbol and betmode information, backend information such as file hash values and a configuration file for the optimization algorithm.

The optimization algorithm consumes the lookup table and outputs a copy of the file, but with modified weights. To assist with setting optimization parameters, there are two other files with the prefix `lookUpTableIdToCriteria` and `lookUpTableSegmented`. These files are used to identify which bet-mode sub-type that specific simulation

number belongs to (such as max-wins, 0-wins, freegame entry etc..), and what gametype (usually basegame or freegame) contributes to the final payout multiplier.

Simulation Acceptance Criteria

When setting up the game configuration file each mode is split into different win-criteria. Given a total number of simulations for a given bet-mode, the number of simulations required for each criteria is set using a `quota`, which determines the ratio of the total number of simulations satisfying a particular win criteria.

Following the example used in the Sample Games, the win criteria has been split into the following unique conditions:

1. `0` win amounts
2. `basegame` wins
3. `freegame` scenarios
4. `max-win` scenarios

The purpose of segmenting these game outcomes is to ensure that there are sufficiently many simulations scenarios satisfying a certain criteria. For example if the hit-rate for a max-win is 1% of the available RTP for a game with a 5000x payout would be 1 in 500,000 outcomes. Though if we are only producing 1 Million simulations in total for this mode, we would like to have more than 2 simulations in total which result in the maximum win amount. This reduces the possibility of any players seeing the same outcomes for a specific win amount.

In the aforementioned list `0` dictates that the payout multiplier is ==0 for that simulation number. `basegame` is essentially any basegame spin where the payout is >0 and the `freegame` is not triggered. `freegame` is any scenario where the `freegame` is triggered from the basegame. `max-win` is any outcome where the maximum payout multiplier is awarded.

This segmentation of wins is also used by the [optimization algorithm](#).

Pertinent to this section though, the simulation acceptance criteria is integral to the `repeat`condition implemented in all sample games. When the `GameState` is setup, the acceptance criteria is assigned to a specific simulation number before any simulations are carried out. So simulation 10, for example, is predetermined to be a simulation which triggers a `freegame`.

When the `run_spin()` function is called and the game-round ends, whether or not the simulation is recorded and added to the state overview is partially determined by the final win condition. If the only condition is that the simulation must be a `0` payout, then the `final_win` value is checked. If this condition is satisfied the `self.repeat = False` and the outcome is saved. Likewise if a particular simulation is determined to be `freegame` criteria, at the end of the spin we verify if the freegame has been triggered and accept the simulation result if so. There can be as many conditions are required in the `self.check_repeat()` function. Just be aware that the more stringent the criteria, the longer a simulation will likely take to run. This time can be quite substantial if the required criteria is unlikely to be achieved naturally. For the `max-win` scenarios for example, generally a specifically made reelstrip is used, and the probability if achieving higher multipliers, prizes etc.. is dictated in the bet-mode distribution.

Predetermining Acceptance

While it would be useful to run the simulations first and then assign the distribution criteria afterwards, this can cause issues when multi-threading larger simulation batches. Simulations relating to max-wins for example typically take substantially longer to succeed than say `0` win simulations. This means that all criteria except the max-win are likely to be filled first, leaving the final thread to deal with many or all of the max-win simulations. For this reason, the `quota` in the BetMode distribution conditions is used in conjunction with the total number of simulations.

Game Configuration Files

The GameState object requires certain parameters to be specified, and should be manually filled out for each new game. These elements are all defined in the `__init__` function. Full details of the expected inputs and data-types are given in the config info section.

General aspects of the game setup which should be considered when creating a `game_config.py`are:

Game-types

Several parts of the engine such as win amount verification, special symbol triggers/attributes and win-levels require the engine to know if the current state of the game is in the *basegame* or *freegame*. For example it is common to perform a weighted draw of some value:

```
#Within game config:

self.multiplier_values = {

   "basegame":{1:100, 2:50, 3: 10},

   "freegame":{2:20, 3:50, 5: 20, 10:10, 20:1}}

....

#Within gamestate:
```

```
multiplier = get_random_outcome(self.config.multiplier_values[self.gametype])
```

Typically special rules apply when the player enters a freegame. The configuration file allows the user to specify the key corresponding to each gametype. By default this is set to `basegame` and `freegame` respectively. All simulations will start in the basegame mode unless otherwise specified, and the transition to the freegame state is handled in the default `reset_fs_spin()` function, which is called as soon as the `run_freespin()` function is entered.

Reels

Most games will use distinct reelstrips for different game-types. It is commonplace for game-modes to have multiple possible reels per mode. One method of adjusting the overall RTP of a game is to have a multiple reelstrips with varying RTP, which can be selected from a weighted draw when calling `self.create_board_from_reelstrips()`. Reelstrips are stored as a dictionary in the `self.config.reels` object. The reelstrip key and csv file name should be specified:

```
reels = {"BR0": "BR0.csv", "FR0": "FR0.csv"}

self.reels = {}

for r, f in reels.items():

    self.reels[r] = self.read_reels_csv(str.join("/", [self.reels_path, f]))
```

Reelstrip weightings are required [distribution conditions](). An example of using multiple reelstrips for each gametype can be applied as:

```
conditions={

    "reel_weights": {self.basegame_type: {"BR0": 2, "BR1": 1},
self.freegame_type: {"FR0":5, "FR1": 1}},

},
```

Scatter triggers and Anticipation

Freegame entry from the basegame or retriggers in the freegame should be specified in the format `{num_scatters: num_spins}`,

```
self.freespin_triggers = {

    self.basegame_type: {3: 10, 4: 15, 5: 20},

    self.freegame_type: {2: 4, 3: 6, 4: 8, 5: 10},

}
```

Symbol initialization

A symbol is determined to be valid if the name exists either in `self.paytable` or in `self.special_symbols`. If a symbol that does not exist in either of these fields is detected when loading reelstrips, a `RuntimeError` is raised.

Symbol values

Winning symbols are determined from the `self.paytable` dictionary object in the game configuration. The expected format is:

```
self.paytable = {

    (kind[int], name[str]): value[float],

    ...

}
```

Where `kind` is the number of winning symbols. For cascading games, or other circumstances where multiple winning symbol numbers pay the same about, for example in the [scatter pays example game]() where 13+ symbols pay the same amount, `self.pay_group` can be defined. By then calling `self.paytable = self.convert_range_table(pay_group)` a paytable of the expected format is generated. The format of the pay-group objects (inclusive of both values in the kind-range) is given as:

```
self.pay_group = {
    ((min_kind[int],max_kind[int]), name[str]): value[float],
    ...
}
```

Special symbols

Special symbol attributes are assigned based on names appearing in `self.special_symbols = {attribute[str]: [name[str], ...]}`. Multiple symbols can share attributes and multiple attributes can be applied to the same symbol. Most games will at least have a `wild` and `scatter`attribute. Once the symbol is initialized, the value of the attribute is accessed through `symbol.attribute` or symbol.get_attribute(attribute) [see Symbols for more information]() regarding symbol object structures. By default the attribute is set to `True`, unless otherwise overridden using the `gamestate.special_symbol_functions`, defined in the gamestate override. All valid bet-modes are defined in the array `self.bet_modes = [ ...]` The `BetMode`class is an important configuration for when setting up game the behavior of a game.This class is used to set maximum win amounts, RTP, bet cost, and distribution conditions. Additional noteworthy tags are:

1. `auto_close_disabled`
    - When this flag is `False` (default) the RGS endpoint API `/endround` is called automatically to close out the bet for efficiency. When the bet is closed however, the player cannot resume their bet. It may be desirable in bonus modes for example, to set this flag to `True` so that the player can resume interrupted play even if the payout is `0`. This means that the front-end will have to manually close out the bet in this instance.
2. `is_feature`
    - When this flag is true, it tells the frontend to preserve the current bet-mode without the need for player interaction. So if the player changes to `alt_mode` where this mode has `is_feature = True`, every time the spin/bet button is pressed, it will call the last selected bet-mode. Unlike in bonus games, where the player needs to confirm the bet-mode choice after each round completion.
3. `is_buybonus`

- This is a flag used for the frontend framework to determine if the mode has been purchased directly (and hence may require a change in assets).

For example, the BetMode class for a bonus/buy feature is taken from the sample *lines* game:

```
BetMode(
    name="bonus",
    cost=100.0,
    rtp=self.rtp,
    max_win=self.wincap,
    auto_close_disabled=False,
    is_feature=False,
    is_buybonus=True,
    distributions=[
        Distribution(
            criteria="wincap",
            quota=0.001,
            win_criteria=self.wincap,
            conditions={
                "reel_weights": {
                    self.basegame_type: {"BR0": 1},
                    self.freegame_type: {"FR0": 1, "WCAP": 5},
                },
                "mult_values": {
```

```
                        self.basegame_type: {1: 1},
                        self.freegame_type: {2: 10, 3: 20, 4: 50, 5: 60, 10: 100, 20: 90, 50: 50},
                    },
                    "scatter_triggers": {4: 1, 5: 2},
                    "force_wincap": True,
                    "force_freegame": True,
                },
            ),
            Distribution(
                criteria="freegame",
                quota=0.999,
                conditions={
                    "reel_weights": {
                        self.basegame_type: {"BR0": 1},
                        self.freegame_type: {"FR0": 1},
                    },
                    "scatter_triggers": {3: 20, 4: 10, 5: 2},
                    "mult_values": {
                        self.basegame_type: {1: 1},
                        self.freegame_type: {2: 100, 3: 80, 4: 50, 5: 20, 10: 10, 20: 5, 50: 1},
                    },
                    "force_wincap": False,
                    "force_freegame": True,
```

```
            },
        ),
    ],
),
```

Distribution Conditions

Within each `BetMode` there is a set of `Distribution` Classes which determine the win-criteria within each bet-mode. Required fields are:

1. Criteria
    - A shorthand name describing the win condition in a single word
2. Quota
    - This is the amount of simulations (as a ratio of the total number of bet-mode simulation) which need to satisfy the corresponding criteria. The quota is normalized when assigning criteria to simulations, so the sum of all quotas does not need to be 1. There is a minimum of 1 simulation assigned per criteria.
3. Conditions
    - Conditions can have an arbitrary number of keys. Though the required keys are:
        - `reel_weights`
        - `force_wincap`
        - `force_freegame`

Note that `force_wincap` and `force_freegame` are set to `False` by default and do not have to be explicitly added.
The most common use for the Distribution Conditions is when drawing a random value using the BetMode's built-in method `get_distribution_conditions()`. i.e.

```
    multiplier =
get_random_outcome(betmode.get_distribution_conditions()['mult_values'])
```

Or to check if a board forcing the `freegame`should be drawn with:

```
if get_distribution_conditions()['force_freegame']:

    ...
```

4.
5. Win criteria (optional)
   There is also a `win_criteria` condition which incorporates a payout multiplier into the simulation acceptance. The two commonly used conditions are `win_criteria = 0.0`and `win_criteria = self.wincap`. When calling `self.check_repeat()` at the end of a simulation, if `win_criteria` is not `None`(default), the final win amount must match the value passed.
   The intention behind betmode distribution conditions is to give the option to handle game actions in a way which depends on the (known) expected simulation. This is most clear if for example a simulation is known to correspond to a `max-win` scenario. Instead of repeated drawing random outcomes which are most likely to be rejected, we can alter the probabilities of larger payouts occurring by biasing a particular reelset, weighting larger prize or multiplier values etc..

Config class object

The game-specific configuration `GameConfig`inherits the `Config` super class. This contains all game specifications, many of which will be set manually for each new game

within `GameConfig`. `Config` allows for setting custom `win_levels`, which are returned during win-events and can indicate the type of animation which needs to be played. Additionally the class sets up several path destinations used for writing files and functions to read in and verify reelstrips stored in the `.csv`format.

Events Module Documentation

Overview

The `events.py` module defines reusable game events that modify the `gamestate` and log significant actions. These events ensure proper tracking of game states and facilitate structured client communication.

Functions

`json_ready_sym(symbol, special_attributes)`

Purpose: Converts a symbol object into a dictionary suitable for JSON serialization, including only specified attributes.

Parameters:

- `symbol (object)`: The symbol object to convert.

- `special_attributes (list)`: A list of attribute names to include if they are not `False`.

`reveal_event(gamestate)`

Purpose: Logs the initial board state, including padding symbols if enabled.

`fs_trigger_event(gamestate, include_padding_index, basegame_trigger, freegame_trigger)`

Purpose: Logs the triggering of free spins, whether from the base game or a retrigger event.

Assertions:

- Either `basegame_trigger` or `freegame_trigger` must be `True`, not both.
- `gamestate.tot_fs` must be greater than 0.

`set_win_event(gamestate, winlevel_key='standard')`

Purpose: Updates the cumulative win amount for a single outcome.

`set_total_event(gamestate)`

Purpose: Updates the total win amount for a betting round, including all free spins.

`set_tumble_event(gamestate)`

Purpose: Logs wins from consecutive tumbles.

`wincap_event(gamestate)`

Purpose: Emits an event when the maximum win amount is reached, stopping further spins.

```
win_info_event(gamestate,
include_padding_index=True)
```

Purpose: Logs winning symbol positions and their win amounts, adjusting for padding if enabled.

```
update_tumble_win_event(gamestate)
```

Purpose: Updates the banner for tumble win amounts.

```
update_freespin_event(gamestate)
```

Purpose: Logs the current and total free spins remaining.

```
freespin_end_event(gamestate,
winlevel_key='endFeature')
```

Purpose: Logs the end of a free spin feature and assigns the final win level.

```
final_win_event(gamestate)
```

Purpose: Logs the final payout multiplier at the end of a simulation.

```
update_global_mult_event(gamestate)
```

Purpose: Logs changes to the global multiplier.

```
tumble_board_event(gamestate)
```

Purpose: Logs symbol positions removed during a tumble and their replacements.

Usage Notes

- Each function appends an event dictionary to `gamestate.book['events']`.
- Deep copies ensure that modifications do not affect past event states.
- Events provide structured output suitable for UI updates and analytics.

This module is essential for maintaining a transparent, trackable game state across different game mechanics.

Executables Class Documentation

Overview

The `Executables` class groups together common actions that are likely to be reused across multiple games. These functions can be overridden in `GameExecutables` or `GameCalculations` if game-specific alterations are required. Generally, `Executables` functions do not return values.

Function Descriptions

```
draw_board(emit_event: bool = True) -> None
```

Forces the initial reveal to have a specific number of scatters if bet mode criteria specify it. Otherwise, it generates a new board and ensures it does not contain more scatters than necessary.

```
force_special_board(force_criteria: str,
num_force_syms: int) -> None
```

Forces a board to have a specified number of a particular symbol by modifying reel stops.

```
get_syms_on_reel(reel_id: str, target_symbol: str)
-> List[List]
```

Returns reel stop positions for a specific symbol name.

```
emit_wayswin_events() -> None
```

Transmits win events associated with ways wins.

```
emit_linewin_events() -> None
```

Transmits win events associated with line wins.

```
emit_tumble_win_events() -> None
```

Transmits win and new board information upon a tumble event.

```
tumble_game_board() -> None
```

Removes winning symbols from the active board and replaces them, triggering a tumble board event.

```
evaluate_wincap() -> None
```

Checks if the running bet win has reached the wincap limit and stops further spin functions if necessary.

```
count_special_symbols(special_sym_criteria: str) -> int
```

Returns the number of active symbols of a specified special kind.

```
check_fs_condition(scatter_key: str = "scatter") -> bool
```

Checks if there are enough active scatters to trigger free spins.

```
check_freespin_entry(scatter_key: str = "scatter") -> bool
```

Ensures that the bet mode criteria are expecting a free spin trigger before proceeding.

```
run_freespin_from_base(scatter_key: str = "scatter") -> None
```

Triggers the free spin function and updates the total number of free spins available.

```
update_freespin_amount(scatter_key: str = "scatter") -> None
```

Sets the initial number of spins for a free game and transmits an event.

`update_fs_retrigger_amt(scatter_key: str = "scatter") -> None`

Updates the total number of free spins available when a retrigger occurs.

`update_freespin() -> None`

Called before a new reveal during free spins, resetting spin win data and other relevant attributes.

`end_freespin() -> None`

Transmits the total amount awarded during the free spin session.

`evaluate_finalwin() -> None`

Checks base and free spin sums, then sets the payout multiplier accordingly.

`update_global_mult() -> None`

Increments the multiplier value and emits the corresponding event.

Dependencies

This class relies on multiple external modules, including:

- `src.state.state_conditions.Conditions`
- `src.calculations.lines.LineWins`
- `src.calculations.cluster.ClusterWins`
- `src.calculations.scatter.ScatterWins`
- `src.calculations.ways.WaysWins`

- `src.calculations.tumble.Tumble`
- `src.calculations.statistics.get_random_outcome`
- `src.events.events` (Various event handling functions)

These modules provide necessary game logic, event management, and mathematical calculations for the execution of the class functions.

Usage

This class is designed as a base class and is expected to be extended by game-specific implementations where needed. It ensures core game mechanics, such as board generation, free spin handling, and win event management, are handled in a reusable manner.

GeneralGameState Class Overview

Class: `GeneralGameState`

Description:

The `GeneralGameState` class is an abstract base class (ABC) that defines the general structure for game states. Other game state classes inherit from it. It includes methods for initializing game configurations, resetting states, managing wins, and running simulations.

Constructor:

`__init__(self, config)`

- Initializes the game state with the provided configuration.
- Initializes variables like `library`, `recorded_events`, `special_symbol_functions`, `win_manager`, `criteria`, etc.
- Calls helper methods to reset seeds, create symbol mappings, reset book values, and assign special symbol functions.

Methods:

`create_symbol_map(self) -> None`

- Extracts all valid symbols from the configuration.
- Constructs a `SymbolStorage` object containing all the symbols from the paytable and special symbols.

`assign_special_sym_function(self)` (Abstract Method)

- This method must be overridden in derived classes to define custom symbol behavior.
- Issues a warning if no special symbol functions are defined.

`reset_book(self) -> None`

- Resets global game state variables such as `board`, `book_id`, `book`, and `win_data`.
- Initializes default values for win tracking and spin conditions.
- Resets `win_manager` state.

`reset_seed(self, sim: int = 0) -> None`

- Resets the random number generator seed based on the simulation number for reproducibility.

`reset_fs_spin(self) -> None`

- Resets the free spin game state when triggered.
- Updates `gametype` and resets spin wins in `win_manager`.

`get_betmode(self, mode_name) -> BetMode`

- Retrieves a bet mode configuration based on its name.
- Prints a warning if the bet mode is not found.

`get_current_betmode(self) -> object`

- Returns the current active bet mode.

`get_current_betmode_distributions(self) -> object`

- Retrieves the distribution information for the current bet mode based on the active criteria.
- Raises an error if criteria distribution is not found.

`get_current_distribution_conditions(self) -> dict`

- Returns the conditions required for the current criteria setup.
- Raises an error if bet mode conditions are missing.

`get_wincap_triggered(self) -> bool`

- Checks if a max-win cap has been reached, stopping further spin progress if triggered.

`in_criteria(self, *args) -> bool`

- Checks if the current win criteria match any of the given arguments.

`record(self, description: dict) -> None`

- Records specific game events to the `temp_wins` list for tracking distributions.

`check_force_keys(self, description) -> None`

- Verifies and adds unique force-key parameters to the bet mode configuration.

`combine(self, modes, betmode_name) -> None`

- Merges forced keys from multiple mode configurations into the target bet mode.

`imprint_wins(self) -> None`

- Records triggered events in the `library`and updates `win_manager`.

`update_final_win(self) -> None`

- Computes and verifies the final win amount across base and free games.
- Ensures that total wins do not exceed the win cap.
- Raises an assertion error if the sum of base and free game payouts mismatches the recorded final payout.

`check_repeat(self) -> None`

- Determines if a spin needs to be repeated based on criteria constraints.

`run_spin(self, sim)` (Abstract Method)

- Must be implemented in derived classes.
- Placeholder prints a message if not overridden.

`run_freespin(self)` (Abstract Method)

- Must be implemented in derived classes.
- Placeholder prints a message if not overridden.

`run_sims(self, betmode_copy_list, betmode, sim_to_criteria, total_threads, total_repeats, num_sims, thread_index, repeat_count, compress=True, write_event_list=True) -> None`

- Runs multiple simulations, setting up bet modes and criteria per simulation.
- Tracks and prints RTP calculations.
- Writes temporary JSON files for multi-threaded results.
- Generates lookup tables for criteria and payout distributions.

Summary

- `GeneralGameState` provides a foundation for defining and managing game states.

- It includes methods for configuring symbols, handling wins, recording events, and executing game simulations.
- Certain methods must be overridden in derived classes to customize behavior.

Wallet Manger

When a set of simulations are setup and executed through the `src/state/run_sims()` function, a new instance of the `WinManager` class is spawned. This class is responsible for tracking `basegame` and `freegame` wins for single simulation rounds (when running `run_spin()`), and also for cumulative win amounts for a given `BetMode`.

```
class WinManager:

    def __init__(self, base_game_mode, free_game_mode):

        self.base_game_mode = base_game_mode

        self.free_game_mode = free_game_mode


        self.total_cumulative_wins = 0

        self.cumulative_base_wins = 0

        self.cumulative_free_wins = 0


        self.running_bet_win = 0.0


        self.basegame_wins = 0.0

        self.freegame_wins = 0.0
```

```
        self.spin_win = 0.0

        self.tumble_win = 0.0
```

Cumulative wins

The cumulative win-amounts are useful in the terminal printouts to quickly check the RTP splits for a given multiprocessing thread. These cumulative values are updated each time a simulation is run and successfully passed, within `state.imprint_wins()` basegame and freegame win amounts are updated using `win_manager.update_end_round_wins()`.

`total_cumulative_wins` incorporate wins from all game-types on a single betmode level, while `cumulative_base_wins` and `cumulative_free_wins` track the cumulative win amounts for the basegame and freegame respectively.

Spin-level wins

The `running_bet_win` tracks wins from the basegame and freegame modes and continuously increases during simulation steps. The final `running_bet_win` value will equal the payout multiplier `basegame_wins` and `freegame_wins`are single simulation level parameters which are reset when `run_spin()` is called. These values are subsequently used for the `lookUpTableSegmented` files, which helps to identify the contribution of different game-types to the final payout multiplier.

The `spin_win` property tracks the win for a given `reveal` event. So for example is reset for each spin within a `freegame`. Finally the `tumble_win` property is used for tracking wins where there are consecutive win events within a single reveal, most commonly seen within tumbling/cascading games. We may want to keep track of the cumulative win amount resulting from multiple tumble events to update win-banners or apply multipliers at the end of the sequence.

Update functions

There are several `WinManager` update functions used to update and reset the `spin_win` and gametype wins. The `running_bet_win` property does not need to be called explicitly, nor does the `cumulative_wins` (as this is called when the simulation is accepted and saved). The gametype should be updated explicitly though when the basegame actions have concluded, as well as at the end of each freegame spin (if applicable). This can be seen the sample `gamestate.run_spin()` game files:

```
self.win_manager.update_gametype_wins(self.gametype)
```

Output files

All relevant output files are automatically generated within the `game/library/`directories. If the required sub-directories do not exist, the will be automatically generated.

Books

The primary data file output when simulations are run are the book files. These contain summary simulation information such as the final payout multiplier, basegame and freegame win contributions, the simulation criteria and simulation events. The contents of `book.events` is the information returned by the RGS `play/` API response.

The uncompressed `books/` files are used within the front-end testing framework and should be used to debug events. Only a small number of simulations should be run due to the file size. Compressed book files are what is uploaded to `AWS` and consumed by the RGS when games are being uploaded. Only data from compressed books will be returned from the `play/` API.

Force files

Each bet mode will output a file of the format `force_mode.json`. Every time the `.record()`function is called, the description keys used as input are appended to the file. If the key already exists, the `book-id` is appended to the array. This file is used to count

instances of particular events. The optimization algorithm also makes use of these keys to identify max-win and freegame books. Once all bet mode simulations are finished, a `force.json` file is output which contains all the unique fields and keys.

Lookup tables

The final payout multiplier for each simulation is summarized in the `lookUpTable_mode.csv`. This is the file accessed by the optimization algorithm, which works by adjusting the weights, initially assigned to `1`. There is also a `IdToCriteria`file which indicates the win criteria required by a specific simulation number, and a `Segmented` file used to identify what gametype contributed to the final payout multiplier. Both these additional files are not typically uploaded to the ACP and are instead used for various analysis functions.

Config files

There are three config files generated after all simulations and optimizations are run. `config_math.json` is used by the optimization algorithm and contains all relevant bet mode details, RTP splits and optimization parameters. `config_fe.json` is used by the front-end frame work and contains symbol information, padding reels and bet mode details which need to be displayed to players. `config.json` contains bet mode information and file hash information and used used by the RGS to determine and verify changes to files being uploaded to the ACP.

File path construction

The `OutputFiles` class within `src/config/output_filenames` is used to construct filepaths and output filenames as well as setting up output folders if they do not yet exist.

Game Board

The `Board` class inherits the https://stake-engine.com/docs/math/source-files/state

The `Board` class inherits the `GeneraGameState` (https://stake-engine.com/docs/math/source-files/state) class and handles the generation of game boards. Most commonly used is the `create_board_reelstrips()` function. Which selects a reelset as defined in the `BetMode.Distribution.conditions` class. For each reel a random stopping position is chosen with uniform probability on the range *[0,len(reelstrip[reel])-1]*. For each reelstop a 2D list of `Symbol` objects are created and attached to the GameState object.

Additionally, special symbol information is included (*special_symbols_on_board*) along with the reelstop values (*reel_positions*), padding symbols directly above and below the active board (*padding_positions*) and which reelstrip-id was used.

The is also an *anticipation* field which is used for adding a delay to reel reveals if the number of Scatters required for trigging the freegame is almost satisfied. This is an array of values initialized to `0` and counting upwards in `+1` value increments. For example if 3 Scatter symbols are needed to trigger the freegame and there are Scatters revealed on reels 0 and 1, the array would take the form (for a 5 reel game):

```
self.anticipation = [0, 0, 1, 2, 3]
```

If the selected reel_pos + the length of the board is greater than the total reelstrip length, the stopping position is wrapped around to the 0 index:

```
self.reelstrip[reel][(reel_pos - 1) % len(self.reelstrip[reel])]
```

The reelset used is drawn from the weighted possible reelstrips as defined in the `BetMode.betmode.distributions.conditions`class (and hence is a required field in the `BetMode` object):

```
   self.reelstrip_id = get_random_outcome(

self.get_current_distribution_conditions()["reel_weights"][self.gametype]

    )
```

Specific stopping positions can also be forced given a reelstrip-id and integer stopping values from `force_board_from_reelstrips()`. If no integer value are provided for a reel, a random position is chosen. This function is typically used in conjunction with `executables.force_special_board`, which will search a reelstrip for a particular symbol name and randomly select a specified number of stopping positions, chosen to land on a randomly selected board row.

Additionally the `Board` class handled symbol generation, displaying the current `.board` in the terminal, and retrieving symbol positions and properties as defined in `config.special_symbols`.

Tumbling boards

The `Tumble` class inherits `Board` and handles removing winning symbols from `self.board`and filling vacant positions with symbols which appear directly above winning positions using the properties `reel_positions` and `reelstrip_id`. Examples of applications surrounding tumbling (cascading) events can be found in the `0_0_cluster` and `0_0_scatter`sample games.

The win evaluation functions for the cluster and scatter win-types assign the property `explode = True` to winning symbol objects. A new board is select by scanning the current `self.board`object reel-by-reel and counting the number of symbols which satisfy `sym.check_attribute("explode")`. This same number of symbols is then appended, counting backwards from the initial `self.reel_positions` values. If padding symbols are used, the symbol stored in `top_symbols` will be used to fill the first vacated position.

Line wins evaluation

The `LinesWins` object evaluates winning symbol combinations for the current `self.board` state. Generally 3 or more consecutive symbols result in a win, though these specific combination numbers and payouts can be defined in:

```
config.paytable = {(kind[int], symbol[string]): payout[float]}
```

In order to identify winning lines, line arrays must be defined in:

```
config.paylines = {

    0: [0,0,0,0,0],

    1: [0,1,0,1,0],

        ...

    }
```

in the `.paylines` dictionary, the key is the line-index and the value is an array dictating which rows result in a winning combination. Like symbols are matched and if the key `(kind, name)` exists in `self.paytable`, the corresponding win is evaluated.

Custom keys used to identify wild attributes and symbol names can be explicitly set and will default to `"wild"` and `"W"` unless otherwise specified. In the case of `(kind, "W")` existing in `self.paytable`, the base payout value is checked against the `(kind, sym)` where *sym* is the first non-wild. If for example the payline `[0,0,0,0,0]` has the symbol combination `[W,W,W,L4,L4]`, resulting in wins `(3,"W")` or `(5,"L4")`. We compare both outcomes and determine that the three-kind Wild combination has a larger payout. Therefore we only take the first three symbols as the winning combination. Note that the sample lines calculation provided will only take into account the base-game wins. If the game is more complex, such as having multipliers on symbols, the final payout amount may need to be handled separately when deciding which winning combination to use. One common approach to dealing with this is to only define the Wild symbols to pay when there is a complete line (so only 5-kind Wilds would pay for a board of this size).

The `get_lines()` evaluation function returns all win information including the winning symbol name, winning positions, number of consecutive matches and win amounts. The

`meta` information also includes symbol and global multiplier information, as well as the index of winning lines as defined in `config.paylines = {index: [line], ... }`.

Ways wins evaluation

The `WaysWins` object evaluates winning symbol combinations for the current `self.board` state. Generally 3 or more consecutive symbols result in a win, though these specific combination numbers and payouts can be defined in:

```
config.paytable = {(kind[int], symbol[string]): payout[float]}
```

The ways calculation will search for like-symbols (or Wilds) on consecutive reels. The maximum number of ways is determined from the board size: `max_ways = (num_rows)^(num_columns)`. Note: the ways calculation does not account for Wild symbols appearing on the first reel.

The Ways evaluation takes also takes into account multiplier values attached to symbols containing the `multiplier` attribute. Unlike lines calculations where multiplier values are added together for symbols on consecutive reels, the total number of ways is instead multiplied by the multiplier value. Leading to the payout amount to grow substantially more quickly. So for example given the board:

```
L5 H1 L4 L4 L4

L1 H4 L3 H2 L4

H1 H1 H1 L3 H3
```

If there is a multiplier value of, say 3x on the `H1`symbol on reel 3, the total ways for symbol `H1` is `(3,H1)` pays:

```
(1) * (2) * (3) = 6 ways
```

The `return_data` will include all winning symbol names, number of consecutive like-symbols, winning positions and total win amounts for each unique symbol type. the `meta` tag will additionally include the total number of ways a symbol wins, which will range from `1` to `(num_rows)^(num_columns)` and and additional symbol and/or global multiplier contributions.

Scatter Pays

Scatter-pays (pay-anywhere) games award wins based on the total number of like-symbols appearing on the game board. Symbols do not have to be arranged in any order. Typically a minimum of 8 like-symbols (or Wilds) are required to count as a win, though these values can be defined in the `GameConfig` class. Since it is possible for up to `num_rows * num_columns`winning symbols to occur, it is common to define a particular payout range. For example `8-kind`pays `p1` `9-kind` to pay `p2`, `10-12` kind to pay `p3` and `12+` symbols pay `p4` etc… Instead of manually including all possible pay combinations in `config.paytable` there is a `convert_range_table()` function in the Config class which takes in a symbol range, name and payout amount which is used to generate all `config.paytable` entries. This pay group should be of the format:

```
paygroup = {

    ((min_combination[int], max_combination[int]), name[str]) : payout[float],

    ...

}
```

Ranges defined in `min_combination` and `max_combination` are inclusive, so for example if the `8-kind` payout for symbol `H1` pays `10x`, this would be written as: `((8,8),H1): 10`.

Often (though not always) Scatter pays games are also cascading/tumbling. Within the [scatter sample game]() for example, while there are still winning combinations, the board is tumbled, wins are evaluated for the new board, the wallet manager is updated and relevant events are emitted:

```
while self.win_data["totalWin"] > 0 and not (self.wincap_triggered):

    self.tumble_game_board()

    self.win_data = self.get_scatterpay_wins(record_wins=True)

    self.win_manager.update_spinwin(self.win_data["totalWin"])

    self.emit_tumble_win_events()
```

The Scatter pay evaluation function also checks for `multiplier` and `wild` attributes attached to symbols. Wild symbols can contribute to wins for any number of symbols.

Cluster Pays

Cluster games award wins when there are sufficiently many neighboring like-symbols. Neighbours must share the same reel or row, where diagonal connections do not count towards the cluster size. A minimum of 5 like-symbols is typical, though this can be defined in `GameConfig` class. Since it is possible for up to `num_rows * num_columns` winning symbols to occur, it is common to define a particular payout range. For example `5 kind` pays `p1` `6-7 kind` to pay `p2`, `8-10 kind` to pay `p3` and `12+` symbols pay `p4` etc… Instead of manually including all possible pay combinations in `config.paytable` there is a `convert_range_table()` function in the Config class which takes in a symbol range, name and payout amount which is used to generate all `config.paytable` entries. This pay group should be of the format:

```
paygroup = {
    ((min_combination[int], max_combination[int]), name[str]) : payout[float],
    ...
}
```

Ranges defined in `min_combination` and `max_combination` are inclusive, so for example if the `5-kind` payout for symbol `H1` pays `10x`, this would be written as: `((5,5),H1): 10`.

Often (though not always) cluster pays games include a tumbling mechanic. Within the [cluster sample game]() for example, while there are still winning combinations, the board is tumbled, wins are evaluated for the new board, the wallet manager is updated and relevant events are emitted:

```
while self.win_data["totalWin"] > 0 and not (self.wincap_triggered):
    self.tumble_game_board()
    self.win_data = self.get_cluster_data(record_wins=True)
    self.win_manager.update_spinwin(self.win_data["totalWin"])
    self.emit_tumble_win_events()
```

Clusters are found using a Breath First Search (BFS) algorithm. Wild attributes can be set (`wild`is the default value). Wild symbols can contribute to multiple clusters, including those formed by different symbols.

Symbol structure

Symbols are handled as their own distinct class objects. Based only off a symbol name, several useful attibutes are assigned to the object based on if the symbol name appears in in the `config.paytable` or `config.special_symbols` fields.

```
class Symbol:

    def __init__(self, config: object, name: str) -> None:

        self.name = name

        self.special_functions = []

        self.special = False

        is_special = False

        for special_property in config.special_symbols.keys():

            if name in config.special_symbols[special_property]:

                setattr(self, special_property, True)

                is_special = True


        if is_special:

            setattr(self, "special", True)


        self.assign_paying_bool(config)
```

When a new game-board is drawn, a 2D array of symbol objects are generated. At a minimum, the symbol will have the attributes:

- Name
    - [string] shorthand name, typically 1 or 2 letters
- special_functions

- Within the `GameStateOverride` class, special functions can be applied to a symbol as soon as the object is created. This is done through the abstract function, for example:

```
def assign_special_sym_function(self):
    self.special_symbol_functions = {
        "W": [self.assign_mult_property],
    }
def assign_mult_property(self, symbol):
    multiplier_value = get_random_outcome(
self.get_current_distribution_conditions()["mult_values"][self.gametype]
    )
    symbol.assign_attribute({"multiplier": multiplier_value})
```

`assign_special_sym_function()` is called when the `GameState` is initially created. In this example, we are assigning a multiplier value to any new wild ('W') which is created. Any action defined within `self.special_symbol_functions` with the format `{<name>: @callable_func}` will be assigned to the `special_functions` property.

- is_special
  - This property is assigned as `False` by default unless the name appears as a value within `config.special_symbols`
- special_property
  - Properties appearing in `config.special_functions = {'property': [name]}` are set to `True` by default.
- assign_paying_bool()

- This function assigns the properties `is_paying` and `paytable`. If the symbol name appears in `config.paytable` `is_paying` is set to `True` and the relevant paytable values are assigned to `paytable`. Otherwise these values are set to `False` and `None` respectively.

Symbol Attributes

In addition to the application of `special_functions`, attributes are an important characteristic of symbol objects, particularly for checking if there are any special symbols on the game-board which require additional actions. For example if we want to check if a given symbol has a `prize` or `multiplier` attribute:

```
if self.board[reel][row].check_attribute('prize','multiplier'):

    ...
```

The `check_attribute` function will return a `boolean` value if the given attribute exists and its value is not `False`. I.e.:

```
if symbol.check_attribute('prize'):

    win += symbol.get_attribute('prize')
```

Furthermore we can assign properties to a symbol using the `assign_attribute` method. As an example, if we have a game where we have a special symbol denoted by the `enhance` tag. Where the effect of this symbol is to add a `multiplier` value to any active `Wild` symbols. In the `gamestate` we could preform the following actions:

```
if len(self.special_symbols_on_board['enhance']) > 0:

    for sym in self.special_symbols_on_board[wild]:

        mult_val = get_random_outcomes(self.config.mult_values[self.gametype])
```

```
        self.board[sym['reel']][sym['row']].assign_attribute({'multiplier',
mult_val})
```

Active Game Board

The active game-board is created as a 2D array of symbol objects. Each object within the array creates a new object instance.

Displaying the board

The board can be displayed by calling the `print_board()` method in the `Board` class, which will display a correctly orientated printout of all symbol names

```
self.print_board(self.board) ->



L5 L3 L4 L4 L4

L3 H4 L3 H1 L4

L3 H1 S  L3 H1
```

Active special symbols

When the game board is generated any symbols appearing in `config.special_symbols = {'property' : [symbols, ..]}` will be appended to the gamestate property `special_symbols_on_board = {'property': [{'reel':`

`reel[int], 'row': row[int]}]}`. This property is particularly useful for checking aspects such as freegame entry conditions:

```
if len(self.special_symbols_on_board['scatter']) >= min_scatter:

    self.run_freespin_from_base()
```

Care should be taken to update any new symbols which may appear on the board either from cascading events or through the application of some special action, such as removing symbols from the game board. If custom functions are being used which involve altering active symbols, the method `get_special_symbols_on_board()` from the `Board` class should be invoked.

Tumbling the board

For cascading games (such as the Scatter and Cluster example games), winning symbols are removed from the board and symbols above *tumble* down to fill these vacant positions. Winning symbols are assigned the attribute `explode`. Subsequently when the `tumble_board()` method is called from the `Tumble` class,

Top/bottom symbols

In the `config` class, there is a boolean option `include_padding`. This is to account for games where it is desirable for the player to see the symbols immediately above/below the active board. Usually this is displayed as a symbol being partially in-frame. If this flag is set to true, the row indexing for the active game board will start at `row=1`, where `row 0` is the `top_symbol` and `row len(board) + 1` is the `bottom_symbol`. The top and bottom symbols are included in the `board reveal` event. Within the gamestate these symbols are stored as:

```
self.top_symbols = [s1, s2, ....]

self.bottom_symbols = [s1, s2, ....]
```

Note that for cascading/tumbling games, the top symbol is preserved during the tumble.

Toggle theme

Win calculations

There are several built-in win methods included in the engine:

1. Lines pays
2. Ways pays
3. Cluster pays
4. Scatter pays

Irrespective of the win method applied, win information is stored in the gamestate object win_data:

```
win_data = {
```

```
    'totalWin': [float],
    'wins': [List[Dict]]
}
```

This initialized `win_data` structure is the return value for all provided win calculation functions. If using the predefined win events, the dictionary items within `wins` must contain the “position” key to account for modifying the row number if needed for the padding symbols. All wins information for the current game board should be included in this structure. Such as all winning symbol combinations, win amounts and positions. The built-in functions also include a `meta’ key which includes any additional information which the front-end may need to display. For the win-lines, as an example this appears as:

```
'wins': {
    'symbol': 'H1',
    'kind': 5,
    'win': 300,
    'positions': [{'reel':1, 'row':1}, ...],
    'meta':{
        'lineIndex': 12,
        'multiplier': 10,
        'winWithoutMult': 30,
        'globalMult': 1,
        'lineMultiplier': 10
```

```
    }
}
```

This additional information includes any symbol or global multiplier values applied, the base win amount, and the `lineIndex`, as defined in `config.paylines = {[], ...}`

Multiplier methods

For generality all win methods utilize functions from the `wins/multiplier_strategy` file. By calling `apply_mult()` with a specified strategy (`global`, `symbol`, `combined`), base win amount and winning symbol positions, total win amounts are returned inclusive of any global multipliers or symbol multipliers. By default, if the `combined` or `symbol` strategy is used, multiplier values are added together from winning symbol positions, where the symbol object contains the `multiplier` attribute.

Overlay values

The cluster and scatter pay sample games, there is an `overlay` key included ine `win_data`“meta” tag of the structure:

```
'meta': {
    ...
    'overlay': {'reel': [int], 'row': [int]}
}
```

This position is calculated as the board position closest to the centre-of-mass of winning clusters.

Wallet manager

When writing game logic, the intent is to have a clear separation of logic, events and wins for clarity. The wins are all handled through a `WalletManager` class, which will handle outcomes from single spins while also keeping track of total cumulative win amount for RTP calculations, as well as which gametype the wins arise from.

This can be seen in a typical gamestate `run_spin()` function where wins are calculated, the wallet is updated and corresponding win events are emitted:

```
self.win_data = self.get_lines()

self.win_manager.update_spinwin(self.win_data["totalWin"])

self.emit_linewin_events()
```

Within a single spin there are wallet manager values associated with:

1. `spin_win`
    - This is the win associated with a specific `reveal` event. If the freegame is entered, this value is reset for each new spin.
    - Updated using `wallet_manager.update_spinwin(win_amount: float)`
2. `running_bet_win`
    - This is the cumulative win amount for a simulation. The final value which the `running_bet_win` is updated with should match the `payout_multiplier`for that simulation.
    - This value is automatically updated with the `wallet_manager.set_spinwin(win_amount: float)` method.

3. `basegame_wins`/`freegame_wins`
    - This value is updated once all basegame actions are completed, or at the end of each freegame spin.
    - Updated using `wallet_manager.update_gametype_wins(self.gametype)`
    - Important! As part of the final payout verification *self.final_win* and *sum(self.basegame_wins + self.freegame_wins)* must match. If these two payouts do not match a `RuntimeError` is raised.
    - This is useful for game analysis and applying the correct parameters to the optimization algorithm.
4. Cumulative simulation wins
    - `total_cumulative_wins`, `cumulative_base_wins` and `cumulative_free_wins` wins are updated at the end of each simulation. This value is used to display the runtime RTP for all simulations when printed in the terminal.
    - Updated using `wallet_manager.update_end_round_wins()` within the `imprint_wins` function

Game Event Structures

Events are the JSON objects returned from the RGS `play/` API and make up the vast majority of data with a game's *library*. Events contain all information required by the front-end to display the current state of the game. Anything not contained within or implied by the events cannot be shown to the player. For a typical game this includes, but is not limited to

- Active game-board symbols
- Freespin counters
- Win counters
- Symbol win information

- Multipliers
- Special symbol actions
- …

The events are crucial as all events need to be handled by the front-end. The user is free to determine their event structure, though to follow the example games, all events have the format,

```
event = {

    "index": [int],

    "type": [str],

    "<field_1>": [T],

    ...

    "<field_n>": [T]

}
```

`"index"` keeps track of the current number of events in a simulation, `"type"` is a unique keyword used to identify an event and is generally a one-word description. `"fields"` are strings who's corresponding value can have any data-type, as required. Once constructed, the event is appended to the book, "events" field":

```
gamestate.book.add_event(event)
```

Events are handled separately in the gamestate to game calculations or executables. They are imported explicitly and not attached to the gamestate object. Once the math-engine has made the appropriate board transformation or action, the event should be emitted immediately, as it will provide a *snapshot* of the current state of the game. For example:

```
from src.Events.Events import update_freespin_event
```

```
run_spin():

    ...

    update_freespin_event(self)

    ....
```

These events should be sent anytime new information needs to be communicated to the player.

Custom Defined Events

Every betmode will have a corresponding `force_record_<betmode>.json`. This file records the `book-id` corresponding to a custom defined search key. Anytime `self.record()` is called where

```
def record(self, description: dict) -> None:

    self.temp_wins.append(description)

    self.temp_wins.append(self.book_id)
```

The current simulation number will be appended to the description/key if it exists, otherwise a new dictionary entry is made based on the description passed to the `record()` function. For example, we may want to keep track of how many Scatter symbols caused a freegame trigger. Which will be useful for later analysis to investigate the frequency of any custom defined event. In the freespin trigger executable function for example,

```python
def run_freespin_from_base(self, scatter_key: str = "scatter") -> None:
    self.record(
        {
            "kind": self.count_special_symbols(scatter_key),
            "symbol": scatter_key,
            "gametype": self.gametype,
        }
    )
    self.update_freespin_amount()
    self.run_freespin()
```

This will ultimately output a `force_record_<betmode>.json` with the entries:


```json
[
    {
        "search": {
            "gametype": "basegame",
            "kind": 5,
            "symbol": "scatter"
        },
        "timesTriggered": 22134,
        "bookIds": [
            7,
            12,
```

```
            ....
        ]
    },
    {
        "search": {
            "gametype": "basegame",
            "kind": 6,
            "symbol": "scatter"
        },
        "timesTriggered": 1196,
        "bookIds": [
            9,
            10
            ...
        ]
    },
    ...
]
```


Summary force file

Once all simulations have been completed, a `force.json` file is produced, which contains all unique search fields and keys. The intended use for this file is for prototyping, where a drop-down menu, or something of the sort can be created for all possible search conditions.

Accounting for discarded simulations

The `record()` function does not directly append the key/book-id to the force file. This action is only performed once a simulation has completed and is accepted. This is to ensure that keys/ids are not prematurely added if a simulation is rejected. Therefore keys and corresponding simulation ids are appended to `self.temp_wins` and `self.temp_wins` before being finalized within the `imprint_wins()` function within `src/state/state.py`. Keys must be unique, and book-ids are not repeated within keys, though the same book-id may appear within several keys.

Various useful functions

Game analytics

The `run` function within `run_analysis.py` is a helper function for analyzing optimized win-distributions. Note: This program assumes a specific format for optimized game lookup tables, as generated by the provided optimization algorithm. Additionally, automatic generation of hit-rates and simulation counts assumes the existence of a `force_record_<mode>.json` file, where wins have been recorded with the keys:

```
'symbol': '<name>',

'kind' : '<num_symbols_in_win>'
```

For example within the `Lines` class we record wins with the format:

```
def record_line(kind: int, symbol: str, mult: int, gametype: str) -> None:

    """Force file description for line-win."""
```

```
    gamestate.record({"kind": kind, "symbol": symbol, "mult": mult, "gametype": gametype})
```

A `.xlsx` file is produced detailing the hit-rates, RTP contributions and number of simulations recorded within of pre-defined win-ranges. Assuming that the `gametype` is recorded, hit-rates for game-types matched to `BetMode.criteria` inputs. This allows for visualizing if win-ranges are occurring in or out of the feature game. This is particularly useful when setting `scale_factor` values within the `GameOptimization.scaling` class.

Valid symbol names are extracted from the `GameConfig.paytable` component. Using recorded `kind` and `symbol` elements, hit-rates, simulation counts and average payout multiplier amounts for a given simulation are generated.

Custom search keys can be passed to the `run()` function, providing the hit-rates for specific events within the `gamestate.record()` function.

Analysis

Once a lookup table has been optimized it is often useful to analyze the resulting win-distribution, which is a dictionary where the keys are all ordered, unique payouts and the values represent the probability of obtaining this specific payout value.

Misc

Swap lookups

The optimization algorithm outputs several viable lookup tables with the `<game>/library/optimization_files/` folder. This file provides functions for swapping out weights in the `<game>/library/lookup_tables/lookUpTable_<mode>_0.csv`file/.

Get file hash

Helper functions for printing the SHA256 values of a single file or all non-python files within a directory to console. These values can be compared with SHA values with `config.json` files to check if file contents have been altered.

Sample Games

There are 4 example games included to showcase different win-types and mechanics. All games have a basegame mode (all 1x bet cost) and 1 freegame mode. The expanding wilds game additionally has a *superspin* mode to showcase how prize-values are handled.

Each game-type has a *readme.txt* file with a brief description of game-rules (copied below).

Lines Games

This is an example of a simple lines-game-win

Wilds have multipliers in the freeGame and have the effect of multiplying a given line win the addition multiplier values attached to Wild symbols, only when the multiplier value is > 1.

Basegame:

Scatter Symbols appear on all reels, a minimum of 3 Scatters are needed to trigger the Freegame

FreeGame:

A seperate reelset is used for the freegame Wilds have larger multipliers in the freegame (minimum of 2x) and appear on all reels 2 Scatters are needed to trigger extra spins, appearing only on reels 2,3,4

Notes: Wilds only pay on 5-Kind. If the paytable is chosen such that 3/4 Kind Wilds pay, the line calculation will assign the highest base-win symbols as winning. For example if

there is a 3-Kind Wild is on the same line as a 5-Kind L4, the 3-Kind wild will be chosen, regardless of the multiplier on the final Wild since the base payout 3W > 5L4

Ways Game

Standard ways game with 5-reels and 3-rows.

- 9 paying symbols (H1-H5, L1-L4)
- 1 wild type of Wild symbol
- 1 type of Scatter symbol
- Multipliers on Wilds (in freegame only)
- Wilds do not appear on 1st reel

Basegame

Minimum of 3 Scatter symbols are needed to enter the freegame. Maximum of 1 Scatter per reel.

Freegame rules

Wild symbols have multipliers ranging from 1x to 5x. Multiplier values compound multiplicatively (unlike lines games where multiplier values add)

Cluster-based win game

Clusters of 5 or more like-symbols are removed from the board, and symbols above on the reelstrip fall to fill their place.

Basegame:

Standard tumbling game with Scatter and Wild symbols. Minimum of 4 Scatter symbols are required for freeSpin triggers

Freegame:

Same basegame rule, except grid positions have multipliers. Grid positions start in a ‘deactivated’ state. Once one win occurs, the position is ‘activated’ starting with a 1x multiplier - for every winning cluster, the multiplier value at that position is doubled (up to 512x) There is a global multiplier, which increases by +1 for every freespin and does not reset on each spin A minimum of 3 scatters are required for re-triggers

Notes:

Because of the separation between basegame and freegame types - there is an additional freespin entry check to check of the criteria requires a forced freespin condition. Otherwise, occurrences of Scatter symbols tumbling onto the board during basegame criteria may appear.

Scatter-Pays Game

Summary:

- A 6-reel, 5-row pay-anywhere tumbling (cascading) game.
- 8 paying total (4 high, 4 low)
- 2 special symbols (wild, scatter)

Symbols payouts are grouped by cluster-sizes (8-8), (9-10), (11,13), (14,36)

Basegame:

Minimum of 3 Scatter symbols needed for freegame trigger. 2 freegame spins are awarded for each Scatter.

Freegame rules

Every tumble increments the global multiplier by +1, which is persistent throughout the freegame The global multiplier is applied to the tumble win as they are removed from the board After all tumbles have completed: multiply the cumulative tumble win by multipliers on board (multipliers on board do not increment the global mult) If there is a multiplier symbol on the board, this is added to the global multiplier before the final evaluation

Notes

Due to the potential for symbols to tumble into the active board area, there is no upper limit on the number of freegame that can be awarded. The total number of freegame is 2 * (number of Scatters on board). To account for this the usual 'updateTotalFreeSpinAmount' function is overridden in the game_executables.py file.

Event descriptions

"winInfo" Summarizes winning combinations. Includes multipliers, symbol positions, payInfo [passed for every tumble event] "tumbleBanner" includes values from the cumulative tumble, with global mult applied "setWin" this the result for the entire spin (from on Reveal to the next). Applied after board has stopped tumbling "seTotalWin" the cumulative win for a round. In the base-game this will be equal to the setWin, but in the bonus it will incrementally increase

Expanding Wilds Lines + Superspin mode

- 5-reel, 5-rows
- 15 paylines
- 9 paying symbols
- 1 type of Wild
- 1 type of scatter

Superspin mode, costing 25x. This mode is independent, with no freegame entry.

- 1 *dead* symbol (1)
- 1 *prize* symbol

basegame

Standard lines games rules with Wilds paying on 3, 4 and 5-kind

freegame

1 Wild can initially appear on each reel. Symbol then expands out to fill all active rows. Expanded symbol is sticky and persistent for all remaining freegame spins. On each new reveal a random multiplier ranging from 2x - 50x is assigned. No retriggers in freegame.

superspin

This is a *hold em'* style game. The player can purchase a spin for 25x, and starts with 3 *lives*Each time a prize symbol lands on the board, the 3 available spins reset. Prizes are sticky and evaluated once the player has no new spins remaining.

This game has a purchase-only 'super-spin' mode. This mode can only be activated through a buy menu and cannot be accessed using Scatters like bonus-games

Optimizing win distributions with iterative weighted sampling

A discussion of how the provided optimization algorithm operates can be viewed by [downloading this paper](#).

The aforementioned algorithm is implemented in the Rust programming language, this program compiles down to a binary executable. If the program is being run for the first time, or if there are modifications made to the *main.rs* file, the binary should be rebuilt using:

```
cargo build --release
```

Setting up optimization parameters

The optimization algorithm parameters can be setup and passed within the *run.py* file Game-specific parameters should be set using the OptimizationSetup class. This Class takes as input the game configuration class and appends *opt_params*. This is a dictionary where the keys are the betmode names and have the required inputs:

```
opt_params = <mode_name> : {

    "conditions": ...

    "scaling": ...

    "parameters:

}
```

Each key has a corresponding construction class within `optimization_algorithm/optimization_config.py`

Conditions

The `conditions` key has the setup class `ConstructConditions`. This key separates out specific simulation numbers which the optimization algorithm is applied to. The

optimization program requires knowing what RTP to optimize a subset of solutions to. This is generally separated out into events where it is desirable to control the frequency of such an event occurring. Such as freegame, max wins or 0-win hit-rates. For each of these win types, we need to have a well defined RTP, meaning that we need 2 of the 3 variables, *RTP*, *average wins*, *hit-rates*. You will notice that for the *0 win* conditions in the sample game the hit-rate is undefined (*x*), this is allowed because it is a free-variable. Since all hit-rates of all win-types must sum to be exactly 1, we are able to deduce the hit-rate using 1 - (sum of all other win-type allocations).

IMPORTANT: The order of the *conditions* keys matters, as the simulation ids corresponding to each of these keys must be exclusive. The optimization tool reads these conditions entries in order and assigns the corresponding simulation-ids to each key before removing them from the available pool of simulations. So for example, a *wincap* simulation will mostly likely also correspond to a *freegame* simulation, therefore *wincap* must be called first.

Scaling

We are able to bias particular win-ranges within the optimization program. We initially generating our trial distributions, we can artificially increase or decrease the the Gaussian weights within this range by a particular scale factor. We can also assign a probability of these weights being assigned for each distribution created. Note that biasing particular ranges by a significant amount can be lead to a lower likelihood of a randomly assigned distribution being accepted, so its effect should be used carefully.

Parameters

This input is used to construct a setup file red by the optimization tool. It defines the number of distributions to trial before combination, minimum and maximum mean-to-median distribution scores to control volatility as well as the number of simulated test spins to run in order to rank viable distributions.

Executing optimization script

Once the game specific `OptimizationSetup`class is constructed, a `math_config.json` file is generated containing all relevant game parameters in conjunction with a

`setup.txt` file detailing simulation setup optimization parameters, handled with the `OptimizationExecution` class. Within the `run.py` file we can specify which game modes we would like to optimize and directly run the Rust binary using:

```
optimization_modes_to_run = ["base", "bonus"]

OptimizationExecution().run_all_modes(config, optimization_modes_to_run,
rust_threads)
```

In order to select a result, a random integer value is selected using Golang's *crypto/rand* standard library package to generate a value on the closed interval $[0, \texttt{sum(LUT weights)} - 1]$. This weighted draw method means that we are able to explicitly calculate the Return To Player (RTP) for a game by computing the dot-product of the payout and weight,

$$\mathcal{RTP} = \frac{1}{W} \sum_{i=1}^{N} w_i p_i, \tag{1}$$

where w_i is the weight of a specific simulation, p_i is the payout multiplier for this simulation number and W is the sum of the weights for all possible outcomes.

2 Optimization Algorithm

The algorithm described here is capable of balancing win distributions of finitely constructed mathematical models. When results are initially simulated, all possible outcomes are assigned a weight of '1' such that any unique simulation is equally likely to be chosen. The purpose of this algorithm is to reassign weights for each unique payout such that the discrete set of solutions result in a weighted distribution with the correct RTP, while biasing outcomes to make payouts within a specified range more desirable. Because of this assignment, the initial lookup-table does not have to be balanced (you can more or less pass any RTP lookup table to it and it will still output the desired RTP). In practice, the idea is that the starting point is relatively close to the desired RTP, in order to avoid assigning excessive weighting to any single outcome. If the RTP is far too high, when playing the game the player may notice situations where a board appears 'primed' but does not result in a win. For example, seeing very high multipliers or prizes that then fail to connect.

2.1 Evolutionary Process

The core of this optimization process is the balancing of over- and under-paying distributions, ensuring that the final combined distribution meets the desired Return to Player (RTP). The process can be visualized as blending two opposing forces to reach equilibrium. Ultimately, this process is an evolutionary algorithm that takes in two inputs, one where the RTP is greater than the target value and one where the output is lower than desired. Given these two distributions with the same payout values, a balanced distribution can always be generated by performing a weighted average of the 'over' and 'under' distributions. Given a balanced win distribution ($\mathcal{T}$), a 'positive' distribution ($\mathcal{P}$) where the average win satisfies $\mathcal{P} > \mathcal{T}$, and a 'negative' distribution, $\mathcal{N} < T$ we can solve,

$$\mathcal{T} = x \cdot \mathcal{N} + (1 - x)\mathcal{P}, \tag{2}$$

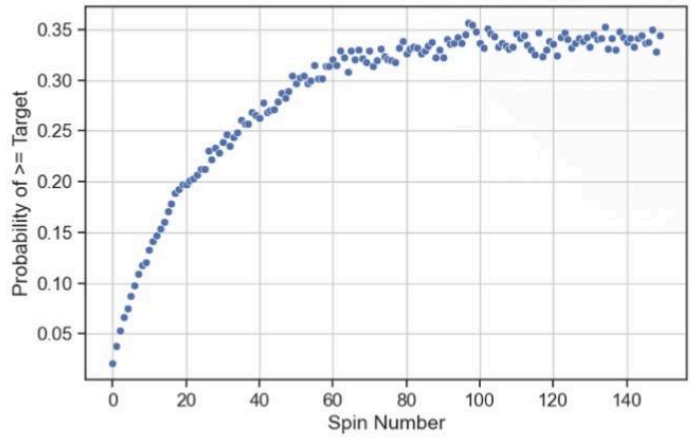


Figure 4: Average probability of a given player being >= 1.0 RTP for a 1x base-game spin using a 0.97 RTP distribution.

performs is 150 spins. We are interested in looking at what the probability is of a player ending an average playing session with a bank balance >= their starting balance (100% RTP). We could investigate any final balance, say 1.5 or 2.0 times their starting balance depending on how we want the game to play. Generally larger values will preference distributions with larger, more sporadic payouts.

```
final_balance = []

for player in range(n_players):
    balance = starting_balance
    spin_balance = []

    for spin in range(N_spins):
        balance -= bet_cost
        payout = weighted_random_draw(payouts, weights)
        balance += payout
        spin_balance.append(balance)

    final_balance.append(spin_balance)
```

For every spin, we can now look at the ratio of players who have a balance >= their starting balance over the total number of players, such as in Figure 4.

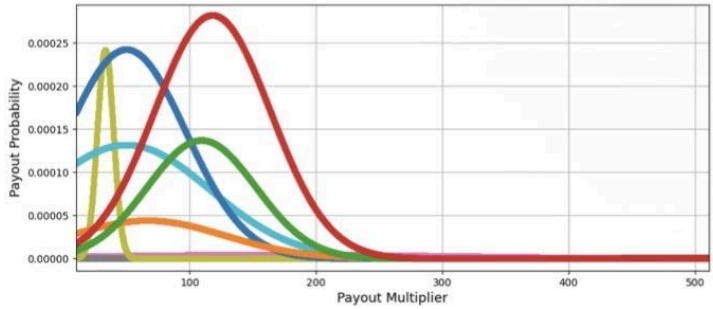


Figure 3: All Gaussian curves constituting the balanced distribution are shown, each with unique amplitudes, standard deviations and means.

will look at a simple measure of volatility by comparing the distribution **mean** and **median** ratios.

The user should define the limits on these values which will depend on how they want the game to *feel*. One method we have utilised is by taking the ratio of the mean payout with the median payout. Where we define the median payout to be the payout multiplier within an ordered win distribution where the cumulative probability is $>= 0.5$. Generally speaking, the larger the value of mean/median, the more volatile the game will feel. As large ratios are indicative of *right-skewed* distributions with more probability being assigned to extreme payouts. Conversely the smaller this value, the more probability is shifted to lower payouts leading to *left-skewed* distributions where more of the distribution probability is clustered around smaller, but more frequent payout amounts closer to the distribution's expected value.

2.3 Ranking Methods

A consideration when selecting win distributions is the macro-scale payout probabilities. We can run simulations for many test players to study the expected player balances after a given number of spins. One strategy discussed here considers distribution volatility by maximising the chances of a player's final balance exceeding some predefined after a fixed number of spins.

To do this we can setup simulated players, typically several thousand, with a large arbitrary starting balance. We then perform a weighted random draw for N spins. After each draw, the players bank balance is updated to reflect the cost of the bet and payout multiplier. The value of N should be selected to represent a realistic number of spins a player is likely to perform. Say, for example, we investigate a game and determine that the average number of spins a player

2.4 Sub-Game-Type Optimization

Much of the power from this optimization algorithm comes from being able to accurately assign hit-rates, probabilities, or average wins to specific game properties. Given that

$$M(x) = R(x) \cdot \frac{1}{p(x)},$$

Where $M(x)$ is the average payout of some event x, R is the RTP contribution of this event occurring and p is the probability of this event. Given two of these three variables, we apply the evolutionary algorithm to these isolated events and recombine the assigned probabilities to create a configurable win distribution.

For example, if we want to consider a game where we have 60% RTP allocated to the *basegame* and 37% RTP allocation to the *freegame*, we can identify which simulations result in a *freegame* entry and apply the evolutionary algorithm separately to both sets of simulations. These simulation events must be mutually exclusive and the problem essentially reduces to optimizing the *basegame* simulations (where there are no *freegame* triggers) to 0.60 RTP and the free-game simulations to 0.37 RTP. Then by enforcing that the sum of all weights are normalised by $\sum_{i=0}^{N-1} w_i = 1$ we can retrieve a lookup table with well defined RTP, hit-rates or average wins for specific events.

In this example we have only considered a base-game and free-game split, but in practice we may want to enforce a specific hit-rate for 0-wins, max-wins or multiple bonus conditions. By applying the process described in Section 2.1 many times, we end up with many balanced *basegame* distributions and equally as many *freegame* distributions. Since all of these sections are balanced, we can iteratively (or randomly) select balanced distributions for each of these sections and test the overall statistical properties after combination. This provides us with many unique payout-weight distributions to test.

2.5 Forced Parameter Scaling

Another powerful feature of this optimization algorithm is the ability to bias selected distributions to preference certain payout ranges (p_{lower}, p_{upper}). We can define a payout range, scaling factor, and probability of applying this factor to a randomly chosen sum of Gaussian curves. Once the combined Gaussian has been constructed and before the distributions are split into the *positive* and *negative* categories we can scale the probabilities within a specified payout range by a constant factor,

$$w_p^* = \mathcal{C} * w_p \quad \text{if } \; p_{lower} <= p <= p_{upper}$$

where $\mathcal{C}$ is a user defined scaling parameter and w_p is the weight associated with the unique payout multiplier (p). This scaling can either be > 1 to increase the assigned probability with the specified range or < 1 to decrease the likelihood of payouts within the chosen range.

2.6 Pseudo-random parameter estimation

Given the brute-force approach there is generally a random-noise term (ξ) added to each parameter to ensure a diverse range of possible solutions,

$$\mu \to \mu + \xi_1,$$
$$\sigma \to \sigma + \xi_2,$$

The principle behind how Gaussian parameters are chosen is outlined briefly below.

2.6.1 Mean Distribution Values: μ

We would like to predominantly select functions whose mean is near that of the target mean, where the likelihood of selecting a μ is proportional to the relative distance $|\mathcal{T} - \mu|$. The mean will be constrained to values $>= 0$ and $<=$ Max(x).

2.6.2 Distribution Standard Deviations σ

The variation in the distribution about mean should not be overly narrow such that the assigned weight is disproportionality applied to a small win range (which would manifest as a narrow spike in the payout-probability distribution plot). We are interested in having an equal number of positive and negative distributions. Since the RTP is particularly sensitive to σ, we generally select a starting value for σ and if we observe significantly more distributions to be positive compared to negative (or *vice-verse*), we vary this value until the number of viable positive and negative distributions balance out.

2.6.3 Distribution Amplitudes $\mathcal{A}$

The selected amplitude should again be proportional to the mean distribution value such that we expect that more significant weighting is assigned to Gaussians whose mean is close to that of the target distribution.

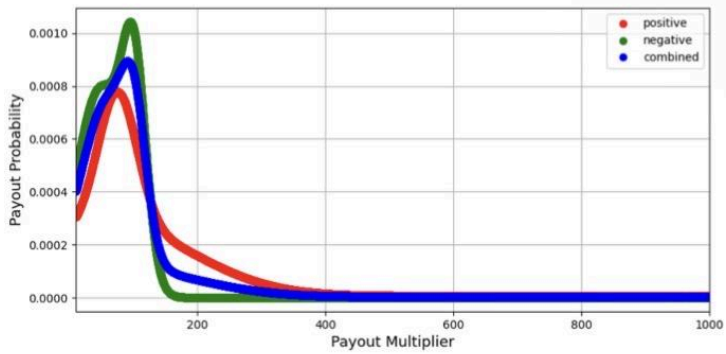


Figure 2: Win distribution balanced to 97% Return To Player (RTP), generated by combining distributions with greater than (positive) and less than (negative) the target RTP for a game with a bet-cost of 100x.

where the weighting, $x \exists (0, 1)$.

$$x = \frac{\mathcal{T} - \mathcal{N}}{\mathcal{P} - \mathcal{N}}. \tag{3}$$

Weighting the sum of the positive and negative distributions by this factor effectively solves the RTP., as demonstrated in Figure 2.

The real crux of the problem comes with how these positive and negative distributions are generated. This is done by brute-forcing random weights for win distributions which satisfy certain statistical properties that well-performing slot games tend to abide by. But generally speaking, we do this by generating many trial distributions which consist of summing Gaussian curves. Recalling the structure of a Gaussian is of the form,

$$\mathcal{G}(x) = \mathcal{A}\frac{1}{\sqrt{2\pi\sigma}} * \exp\big(-\frac{1}{2}(\frac{x-\mu}{\sigma})^2\big), \tag{4}$$

where σ is the standard deviation, μ is the distribution mean, x is the coordinate on the distribution axis (the payout amount here for an ordered win distribution) and $\mathcal{A}$ is some scaling factor.

We start by selecting a random number of Gaussian curves (N) we want to combine (typically between 5 and 15). For each of these distributions we also assign random values for the amplitudes, means and standard deviations. We do this because Gaussians can be conveniently added together to form another

unique curve described by the parameter set:

$$\boldsymbol{\mathcal{A}_i} = [\mathcal{A}_1, \mathcal{A}_2, \ldots, \mathcal{A}_N],$$
$$\boldsymbol{\sigma_i} = [\sigma_1, \sigma_2, \ldots, \sigma_N],$$
$$\boldsymbol{\mu_i} = [\mu_1, \mu_2, \ldots, \mu_N].$$

Once these parameters are selected, we write the form of our trial distribution, with a minimum integer weight of '1' assigned to each payout, as

$$\boldsymbol{\mathcal{G}}(x) = 1 + \sum_{i=1}^{N} \boldsymbol{\mathcal{A}_i} \frac{1}{\sqrt{2\pi \boldsymbol{\sigma}_i}} * \exp\big(-\frac{1}{2}\big(\frac{x - \boldsymbol{\mu}_i}{\boldsymbol{\sigma_i}}\big)^2\big). \tag{5}$$

Leaving out distribution fitness criteria (which will be discussed later), we could simply ask the question of whether $\boldsymbol{\mathcal{G}}(\bar{x})$ has an expected return of $>$ $||$ $<$ the target RTP, where $\bar{x}$ is the set of all possible game payouts. We can continuously create these distributions until we have sufficiently many positive and negative distributions which can iteratively combine. By taking $\boldsymbol{\mathcal{G}_N}(\bar{x})$ as the function producing weights which satisfy, $E\langle\boldsymbol{\mathcal{G}}(\bar{x})\rangle < \mathcal{T}$ and $\boldsymbol{\mathcal{G}_P}(\bar{x})$ as the function resulting in $E\langle\boldsymbol{\mathcal{G}}(\bar{x})\rangle > \mathcal{T}$ with $E\langle\rangle$ is the expectation value of the distribution. We create a balanced distribution by concatenating the Gaussian parameters with amplitudes weighted by Eq. 3 to obtain the new parameter set used to create an optimised distribution,

$$\boldsymbol{\mathcal{A}_T} = \left[\frac{x}{\sum_i \boldsymbol{\mathcal{A}}_{\mathcal{P}i}}[\mathcal{A}_{\mathcal{P}1}, \mathcal{A}_{\mathcal{P}2}, \ldots, \mathcal{A}_N]\right] + \left[\frac{(1-x)}{\sum_i \boldsymbol{\mathcal{A}}_{\mathcal{N}i}}[\mathcal{A}_1, \mathcal{A}_2, \ldots, \mathcal{A}_N]\right]$$
$$\boldsymbol{\sigma_T} = [\sigma_{\mathcal{P}1}, \sigma_{\mathcal{P}2}, \ldots, \sigma_{\mathcal{P}N}] + [\sigma_{\mathcal{N}1}, \sigma_{\mathcal{N}2}, \ldots, \sigma_{\mathcal{N}N}],$$
$$\boldsymbol{\mu_T} = [\mu_{\mathcal{P}1}, \mu_{\mathcal{P}2}, \ldots, \mu_{\mathcal{P}N}] + [\mu_{\mathcal{N}1}, \mu_{\mathcal{N}2}, \ldots, \mu_{\mathcal{N}N}].$$

In practice, these parameters should be selected to correspond to reasonable solutions to slot game mechanics. Such as ensuring that the probability of selecting a value for μ decreases proportionality as the value strays further from the target mean. All we really want to start checking for at this stage is if the distribution generated is above or below the target RTP (as a minimum condition).

By deconstructing the above balanced, combined distributions we can see the individual Gaussian curves which constitute the final win distribution in Figure 3.

2.2 Acceptance Criteria

If we carefully select our Gaussian parameters such that we tend to generate equal numbers of positive and negative distributions, we can efficiently generate many possible trial solutions. Though we want to put some constraints on which are acceptable. The list of requirements can become quite substantial, such as enforcing hit-rate ranges for particular payouts or performing a Mean Square Error analysis on some *ideal* or distribution. To keep things general, we

Brute-Forcing Evolution:
Optimizing Win Distributions with Iterative Weighted Sampling

Carrot Gaming
Declan Armstrong

1 Discrete Outcome Random Draw

In traditional online slot machines, the back-end code is responsible for generating results for the front-end in real time. The Carrot Gaming solution uses a different approach. This is often referred to as a 'lookup table' or pregenerated results approach, which uses a predefined set of outcomes stored in a database. The game result selection process is analogous to a replacement random draw.

Using the random draw approach, a large number of possible outcomes are generated (typically between 500,000 and 1,000,000) independently for each mode within a game. A database stores the instructions for all possible game rounds, along with a lookup table, which is a single file containing a simulation number, weight, and final payout multiplier. Each time a bet is placed, the Remote Game Server (RGS) selects a specific simulation number with a probability proportional to its assigned weight. The game logic for this simulation is then returned in JSON format by querying a database for the given simulation number, which is then passed to the front-end for visualizing gameplay.

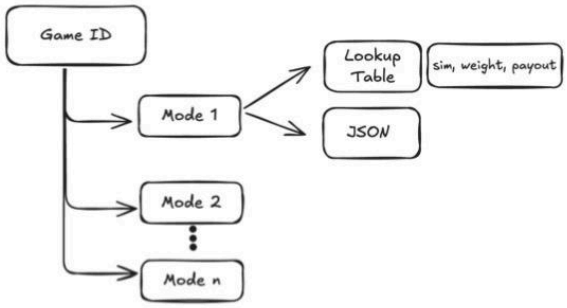


Figure 1: Game information structure